

AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE
WYDZIAŁ FIZYKI I INFORMATYKI STOSOWANEJ

KATEDRA ODDZIAŁYWAŃ I DETEKCJI CZĄSTEK

Praca dyplomowa

Exploitations of Adaptive Hough Transform algorithm for ATLAS experiment inner tracker data for high luminosity running

Wykorzystanie adaptacyjnego algorytmu Hougha dla danych wewnętrznego trackera eksperymentu ATLAS dla danych o wysokiej świeltności

Autor: Piotr Libucha
Kierunek studiów: Informatyka Stosowana
Opiekun pracy: dr hab. inż. Tomasz Bołd

Kraków, 2025

Contents

1 Introduction **6**

1.1 Large Hadron Collider and ATLAS detector 6

1.2 Particle collisions 6

1.3 The problem of this work 7

2 Goals, Assumptions, and Limitations **8**

2.1 Goals 8

2.2 Assumptions 9

2.3 Limitations 9

3 Detailed problem description, data structure and characteristics **10**

3.1 Trajectory characteristics 10

3.1.1 Movement of electrically charged particle in magnetic field 10

3.1.2 Shape and orientation of the trajectory 10

3.1.3 Derivations of formulas without relativistic effects 11

3.1.4 Trajectory origin and parameters 12

3.2 Data structure and characteristics 13

3.3 Particles initial 13

3.4 Spacepoints 15

4 Adaptive Hough Transform **16**

4.1 Hough Transform 16

4.1.1 Hough Transform algorithm 16

4.1.2 Computational and memory complexity 18

4.1.3 Hough space used in this work 19

4.2 Adaptive Approach 21

4.3 Computational and Memory Complexity 24

4.4 Other Characteristics 26

4.4.1 Adaptive Resolution 26

4.4.2 Solution Storage 26

4.4.3 Branching 26

4.4.4 Cache Efficiency 26

4.4.5 Low Cost of Resolution Increase 27

4.4.6 Solution Information Retention 27

4.4.7 Additional Filtering 27

5 The Algorithm **28**
5.1 Terms and Definitions . . . 28
5.2 Splitter . . . 28
5.2.1 Wedges . . . 29
5.2.2 Pole Regions . . . 31
5.2.3 Margins . . . 32
5.2.4 Splitter Regions Used for Most Spacepoints Full Event and Performance Tests . . . 32
5.2.5 Innermost Layer Rejection . . . 33
5.3 Helix Solver . . . 34
5.3.1 Helix Solver Parameters . . . 35
5.4 Additional AHT Filters . . . 38
5.4.1 Layer Hit Check . . . 38
5.4.2 Line Crossing Check . . . 39

6 Solution Quality Tests **42**
6.1 Run Per Particle Initial . . . 42
6.1.1 Efficiency by x- and z-angle . . . 43
6.1.2 Efficiency by z-angle Margin . . . 44
6.1.3 Efficiency by x-angle Margin . . . 45
6.1.4 Efficiency by Number of x-angle and z-angle Ranges . . . 45
6.2 Particles Initial . . . 46
6.2.1 Efficiency by Number of x-angle and z-angle Ranges . . . 47
6.2.2 Number of Solutions by Number of x-angle and z-angle Ranges . . . 48
6.2.3 True Positive Rate by Number of x-angle and z-angle Ranges . . . 49
6.3 Spacepoints Full Event . . . 50
6.3.1 Efficiency, Number of Solutions, and TPR Sensitivity to Hits Threshold and Lines Crossing Threshold by x-angle Range . . . 51
6.3.2 Full Event Test for Selected Per-Region Thresholds . . . 54
6.3.3 Accidentally Correct Solutions Rate . . . 56
6.3.4 Characteristics of Non-Reconstructed Trajectories . . . 57

7 CPU Performance Tests **59**
7.1 Execution Times by Number of x-angle and z-angle Ranges . . . 60
7.2 Execution Times for Custom Per-Region Hits Thresholds . . . 61

8 GPU Implementation **62**
8.1 Technical Challenges Due to OneAPI/SYCL and CUDA . . . 62
8.2 Performance Issues . . . 63

8.3 GPU Implementation Abandonment 63
8.4 GPU Implementation Design 63
8.4.1 Data Organizing Objects 63
8.4.2 Splitter and Solver 64
8.4.3 KernelMemory 64
8.4.4 TaskUsm 64
8.4.5 QueueUsm 64
8.4.6 WorkerUsm 65
8.4.7 IWorkerControllerUsm 65
8.4.8 HelixSolverTask 65
8.4.9 Single Allocation Policy and Resource Borrowing 65

9 Final Considerations for Future Work **66**
9.1 Multidimensional Accumulator 66
9.2 Further Optimizations 66

1 Introduction

1.1 Large Hadron Collider and ATLAS detector

The Large Hadron Collider (LHC) is the world's most powerful particle accelerator, designed to accelerate charged subatomic particles and ions to energies of up to several TeV, with the highest center-of-mass energy so far reaching $6.8TeV$ ($13.6TeV$ collision energy) in proton-proton collisions [22]. The ATLAS (A Toroidal LHC Apparatus) experiment is a general-purpose detector designed to detect high-energy particles emerging from collisions at its center. ATLAS consists of multiple subdetectors. The ones relevant for this work are the components of the Inner Detector: Pixel Detector, Semiconductor Tracker (SCT), and Transition Radiation Tracker (TRT). Their placement is shown in Figure 1.

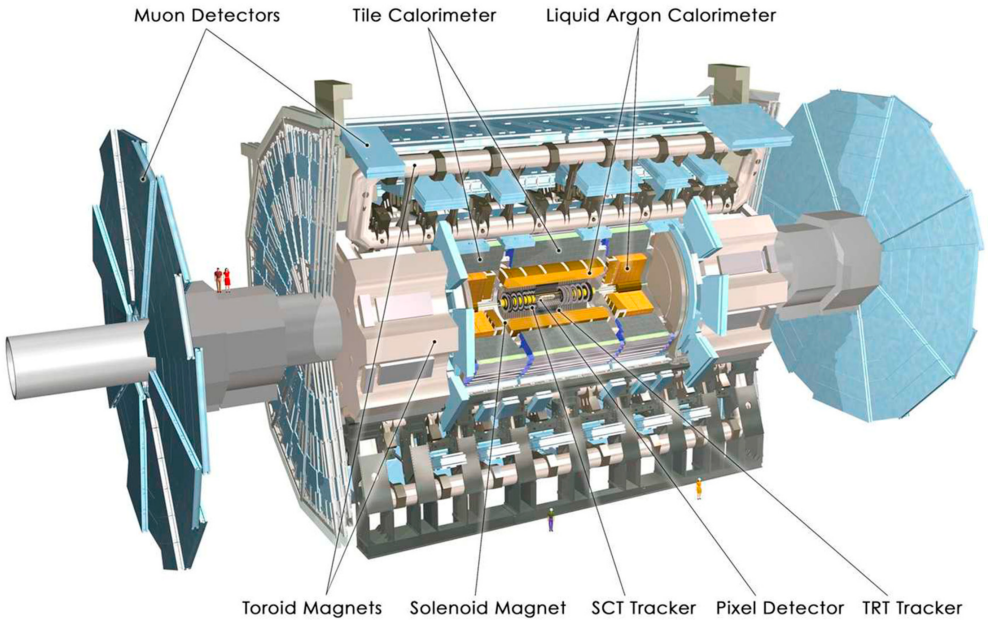


Figure 1: ATLAS detector with the subdetector components. Source: [Gaz24]

1.2 Particle collisions

During a run, two beams of particles are accelerated in opposite directions and collide at the center of the detector. Particles are grouped into bunches. The nominal number of bunches per beam is 2808, and the nominal revolution frequency is $11245Hz$, resulting in a $31.6MHz$ bunch crossing frequency [VM15]. The average number of inelastic particle collisions occurring in the same bunch crossing (known as pile-up) varies between runs. The average pile-up for the values above is $\mu = 27$. However, the original pile-up forecast for LHC was $\mu = 19$ [Apo+15]. The LHC and ATLAS are undergoing an upgrade to the

High-Luminosity Large Hadron Collider (HL-LHC) with the goal of increasing luminosity by a factor of 10, resulting in pile-up of up to $\mu = 200$ [Apo+15] [Luc23].

Each inelastic collision results in the production of a number of high-energy particles. These particles are detected by the ATLAS detector, and their trajectories are reconstructed based on the collected data. The introduction of the HL-LHC will result in a higher number of measurements and therefore make trajectory reconstruction more challenging. Also the inner detector needs will be refurbished for HL-LHC. The aforementioned three sub-detectors, Pixels, SCT and TRT, will be replaced by all-silicon Pixel detectors and Strip detectors, commonly referred to as ITk, covering larger polar angle [ATL23]. The Figure 2 shows a visualization of a bunch crossing with pile-up $\mu = 200$ in the ATLAS Inner Detector.

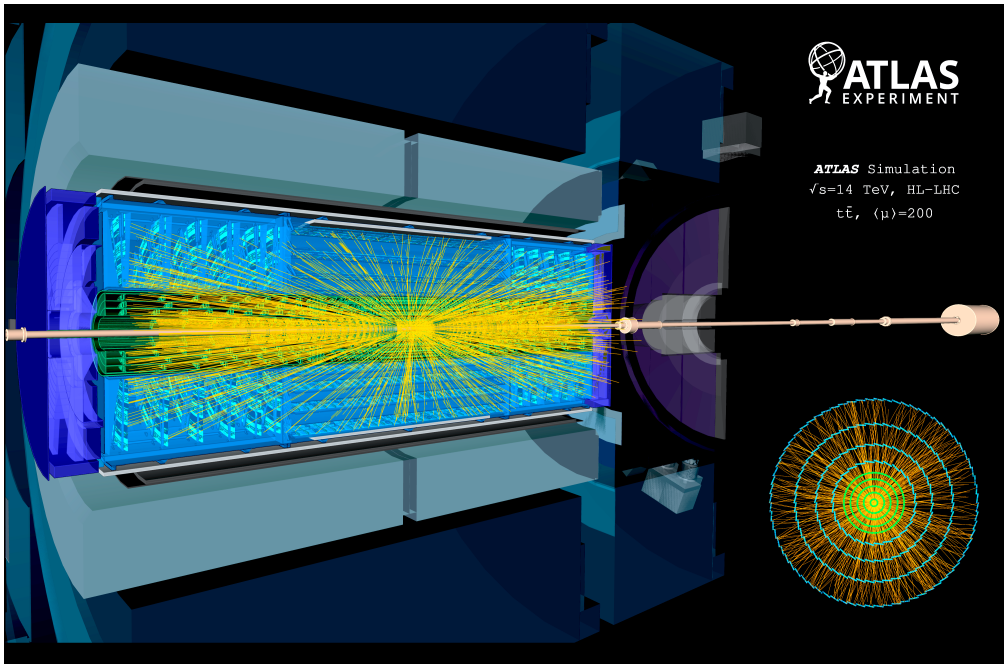


Figure 2: ATLAS HL-LHC bunch crossing simulation with pile-up $\mu = 200$. Source: [23]

1.3 The problem of this work

The objective of this work is to perform the initial reconstruction of trajectories of high-energy particles based on the data collected by the ATLAS ITk. The data provided for this goal are simulated measurements in the ACTS toolkit [Ai+22]. The dataset used for testing consists of 20 bunch crossings (further referred to as events) with pile-up $\mu = 200$. Each event contains, on average, 61983 measurements (also referred to as spacepoints or traces) originating from an average of 4782 particles (referred to as particles initial or truth particles).

2 Goals, Assumptions, and Limitations

2.1 Goals

The main goal is to develop an algorithm that will allow for the preliminary reconstruction of trajectories of high-energy particles. The algorithm should be optimized for three main criteria:

- **Reconstruction efficiency** - efficiency is defined here as the percentage of truth particles whose parameters comply with the assumptions outlined below that match at least one trajectory candidate found by the algorithm.
- **Number of trajectory candidates found** - the number of trajectory candidates found by the algorithm should be as close to the number of truth particles as possible, thereby minimizing the number of false trajectories and duplicates. A ratio of the trajectory candidates found to the number of truth particles close to 10 for low transverse momenta is considered to be a very good result.
- **Computational complexity** - the algorithm should be optimized for computational complexity. This is a critical factor due to the high rate of events that needs to be processed while filtering the data.

Additional desirable properties are:

- parameterization allowing for trade-offs between the criteria,
- adjustable solution accuracy.

The solution, including the algorithm, should be implemented for both CPU and GPU. The implementation shall:

- allow testing of the algorithm efficiency,
- allow testing of the algorithm performance on CPU,
- allow testing of the algorithm performance on GPU with close to full hardware utilization,
- be readable and easy to modify,
- be testable and covered with tests.

Upon achieving the above goals, the algorithm's quality and performance should be evaluated. The evaluation shall provide a comprehensive overview of the algorithm's characteristics in terms of the goals above and provide insights into the algorithm's sensitivity to the parameter choices, available trade-offs, limitations, and improvement opportunities.

2.2 Assumptions

The following assumptions are made:

- This work focuses on the reconstruction of high-energy particles, which are defined as having momentum in the direction perpendicular to the beam axis (transverse momentum) of value $p_T \geqslant 1GeV/c$. Particles with lower momentum are not attempted to be reconstructed in this work, however they contribute to detector signals.
- This work focuses on the reconstruction of particles within a limited absolute pseudorapidity. However, the exact range is not defined. The wider the range with high efficiency, the better.
- This work considers only particles originating directly from collisions.
- Memory complexity is irrelevant as long as long as it does not approach levels that impose limitations on the choice of parameters or hardware requirements above what can be expected from off-the-shelf GPUs or common CPU cluster implementations.
- Results produced by the algorithm can be in any form allowing for easy matching with the truth particles or conversion to the form allowing for such matching.
- Results may be of low precision, since the solution is intended to be a preliminary one, so called seed or proto-track.
- The implementation is tuned to the ATLAS ITk, based on simulation data.

2.3 Limitations

The following limitations are imposed:

- The algorithm is based on the Adaptive Hough Transform (AHT) [IK87].
- The implementation for GPU offloading is based on the OneAPI/SYCL standard.
- The implementation is hardware agnostic, allowing for testing on various CPUs and GPUs.

3 Detailed problem description, data structure and characteristics

3.1 Trajectory characteristics

3.1.1 Movement of electrically charged particle in magnetic field

An electrically charged particle moving in a magnetic field with velocity component v_T perpendicular to the field B experiences a force known as the Lorentz force. The force is always perpendicular to the velocity component v_T and acts as a centripetal force, bending the trajectory of the particle. The Lorentz force is given by the following equation (1):

$$\vec{F_L} = q(\vec{v} \times \vec{B}) \tag{1}$$

$$F_L = q|\vec{v} \times \vec{B}| = qv_T B \tag{2}$$

where:

- $\vec{F_L}$ - Lorentz force,
- q - charge of the particle,
- $\vec{v}$ - velocity of the particle,
- v_T - velocity component perpendicular to the magnetic field $\vec{B}$,
- $\vec{B}$ - magnetic field vector,

3.1.2 Shape and orientation of the trajectory

The Lorentz force formula implies that the force is impacted only by velocity component v_T, therefore if the magnetic field B is uniform, the trajectory of the particle is a helix with pitch proportional to velocity component v_p parallel to the magnetic field as visualized in Figure 3.

The ATLAS ITk is immersed in a strong, close to uniform magnetic field. The field is parallel to the beam axis (z-axis) [[Ale+08]], therefore the helix trajectory projected on the plane perpendicular to the beam axis forms a circle.

Realistic trajectory is disturbed by particle interactions with detector material. In practice, though, for charged particles of p_T above 1 GeV/c the helix is a satisfactory approximation.

3.1.3 Derivations of formulas without relativistic effects

The formula for the magnitude of centrifugal force F_c in the case of circular or helical motion is given by (3):

$$F_c = \frac{mv_T^2}{r} \tag{3}$$

where:

- m - mass of the particle,
- v_T - velocity component perpendicular to the magnetic field $\vec{B}$,
- r - radius of the circle.

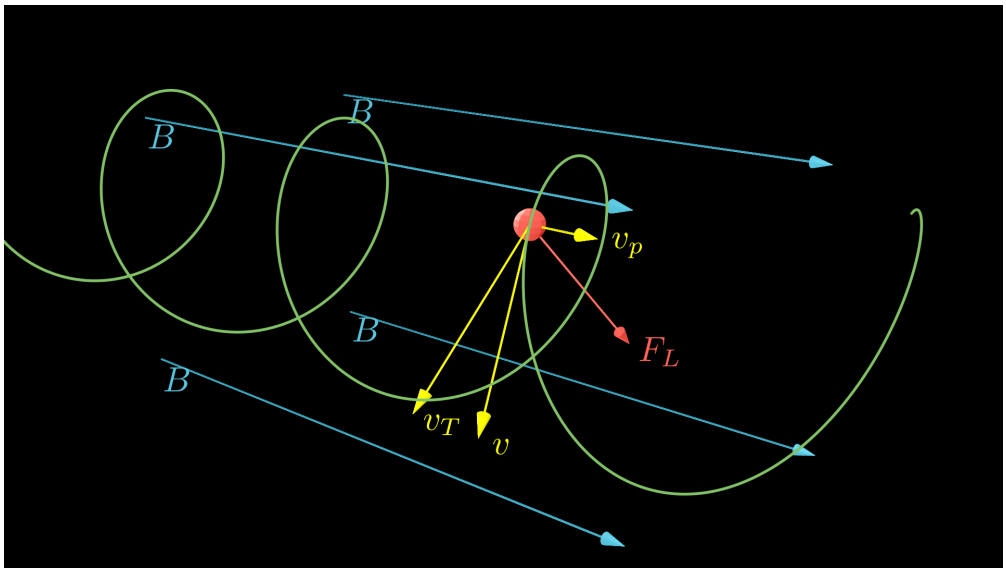


Figure 3: Movement of an electrically charged particle in a magnetic field.

As mentioned above, in our case the Lorentz force plays the role of centripetal force, therefore we can equate it with the centrifugal force (4) and solve for radius r:

$$F_L = F_c \tag{4}$$

$$qv_T B = \frac{mv_T^2}{r} \tag{5}$$

$$r = \frac{mv_T}{qB} \tag{6}$$

Furthermore, we can express mass and velocity component v_T in terms of momentum component p_T in direction perpendicular to the magnetic field (7) by which we obtain:

$$p_T = mv_T \tag{7}$$

$$r = \frac{p_T}{qB} \tag{8}$$

Units used in this work are:

- GeV/c for momentum,
- T for magnetic field,
- e for electric charge,
- mm and m for distance.

Hence we derive the following formula (12) with embedded units:

$$r = \left[\frac{10^9 \frac{1e \cdot 1V}{1c}}{1e \cdot 1T}\right] \cdot \frac{p_T}{qB} \tag{9}$$

$$r = \left[\frac{10^9 \frac{1V}{3 \cdot 10^8 \frac{m}{s}}}{1T}\right] \cdot \frac{p_T}{qB} \tag{10}$$

$$r = \left[\frac{10 \frac{1V}{3 \frac{m}{s}}}{1 \frac{V \cdot s}{m^2}}\right] \cdot \frac{p_T}{qB} \tag{11}$$

$$r = [m] \cdot \frac{10}{3} \cdot \frac{p_T}{qB} \tag{12}$$

3.1.4 Trajectory origin and parameters

Particles leaving traces in the detector originate mostly from a small region, called the interaction region. Only such particles are of interest in this work. As will be shown in the subsequent sections, the interaction region spans about $400mm$ along the z-axis and negligible distance in perpendicular directions, at most a few mm. As mentioned in the assumptions in subsection 2.2, we are interested only in particles with $p_T \geqslant 1GeV/c$. Therefore, assuming particle charge $q = 1e$ and magnetic field $B = 2T$ [[Ale+08]], we can plug values to formula (12) to calculate the minimum radius of the trajectory.

$$r = \frac{10}{3} \cdot \frac{1}{1 \cdot 2} [m] = \frac{10}{6} [m] \approx 1.67 [m] \tag{13}$$

Finally, we can conclude that trajectories we are looking for are small segments of helices originating from interaction point $P \approx (0, 0, z)$, where $z \in [-0.2, 0.2]m$ with radius $r \geqslant 1.67m$ exceeding radius of the ITk $\approx 1m$.

3.2 Data structure and characteristics

Dataset used for development and testing contains two useful types of data - particles initial (truth particles) and spacepoints (measurements) for 20 simulated events with pile-up $\mu = 200$.

3.3 Particles initial

Each initial particle entry provides, among other parameters, the following:

- `eventId` - event identifier,
- `particleId` - a unique identifier of the particle,
- `vx`, `vy`, `vz` - point in space where the particle was created,
- `px`, `py`, `pz` - momentum components,
- `pt` - transverse momentum,
- `p` - momentum magnitude,
- `q` - charge of the particle,
- `eta` - pseudorapidity η.

Relationships between values can be expressed by the following equations:

- $p_x = p_T \cdot \cos(\phi)$
- $p_y = p_T \cdot \sin(\phi)$
- $p_z = p \cdot \sinh(\eta)$
- $|p| = p_T \cosh(\eta)$

where ϕ is the azimuthal angle.

A set of particles initial contains on average 4782 particles. Most of them are created in the interaction region, but the others are included as well. Figure 4 shows the distribution of 5040 particles of event 0 in v_z and v_r space, where v_r is the distance from the beam axis.

Two clusters of origins can be distinguished - one in the interaction region with $|v_z| \leqslant 200$, $v_r \leqslant 0.1$, containing 4657 particles and the other in the region with $v_r \geqslant 0.1$ and $|v_z|$ not limited to 200, containing 383 particles. Particles in the first cluster are grouped

at multiple locations as indicated by higher opacity, which can be assumed to be places where collisions occurred generating multiple subatomic particles. The second cluster is more spread out and is more likely to contain particles generated in other processes such as particle decay. Particles are not grouped as in the case of the first cluster. Note that some of them were created close to or beyond the outer edges of the Inner Detector ($r > 1m \vee |v_z| > 3m$). This might have slight impact on perceived efficiency of the solution.

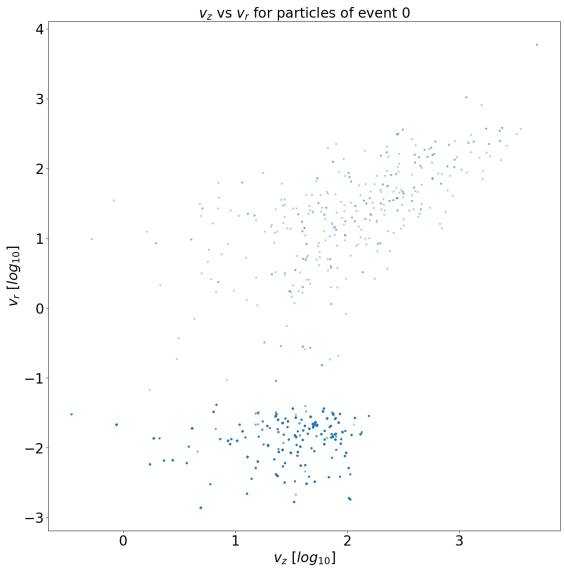


Figure 4: Distribution of v_r and v_z for particles of event 0.

The distribution of particles in the zy plane is non-uniform. Higher density is observed in polar regions (high absolute η) and lower density in the central region. In the xy plane the distribution is uniform. Figure 5 visualizes a subset of particles' momenta directions.

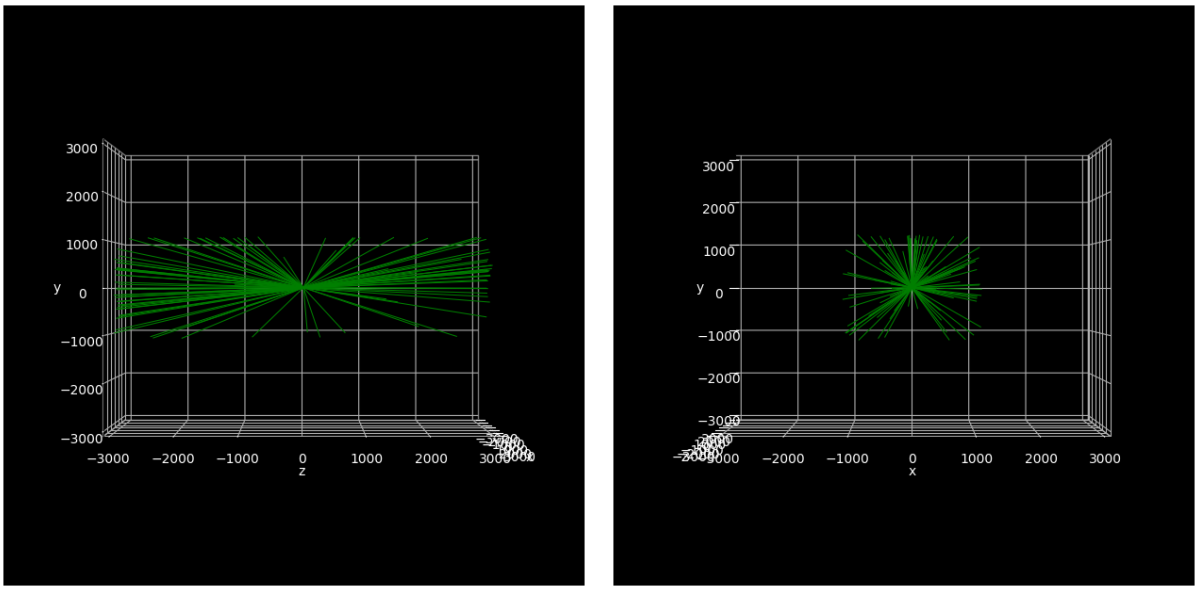


Figure 5: 100 particles' momenta directions in zy and xy planes.

3.4 Spacepoints

Each spacepoint entry provides, among other parameters, the following:

- `eventId` - event identifier,
- `measurementId` - a unique identifier of the measurement,
- `x`, `y`, `z` - position of the measurement.

A set of spacepoints contains on average 61983 measurements. The density of spacepoints is highest in the layers of the Pixel Detector and decreases towards the outer edges of the Inner Detector. The density of measurements increases with $|\eta|$ and local detector resolution. The density drops with the distance from the interaction point. As can be seen in Figure 6 visualizing measurements of a single event, the density of spacepoints is high enough to clearly distinguish between the layers of the Inner Detector and infer their shapes.

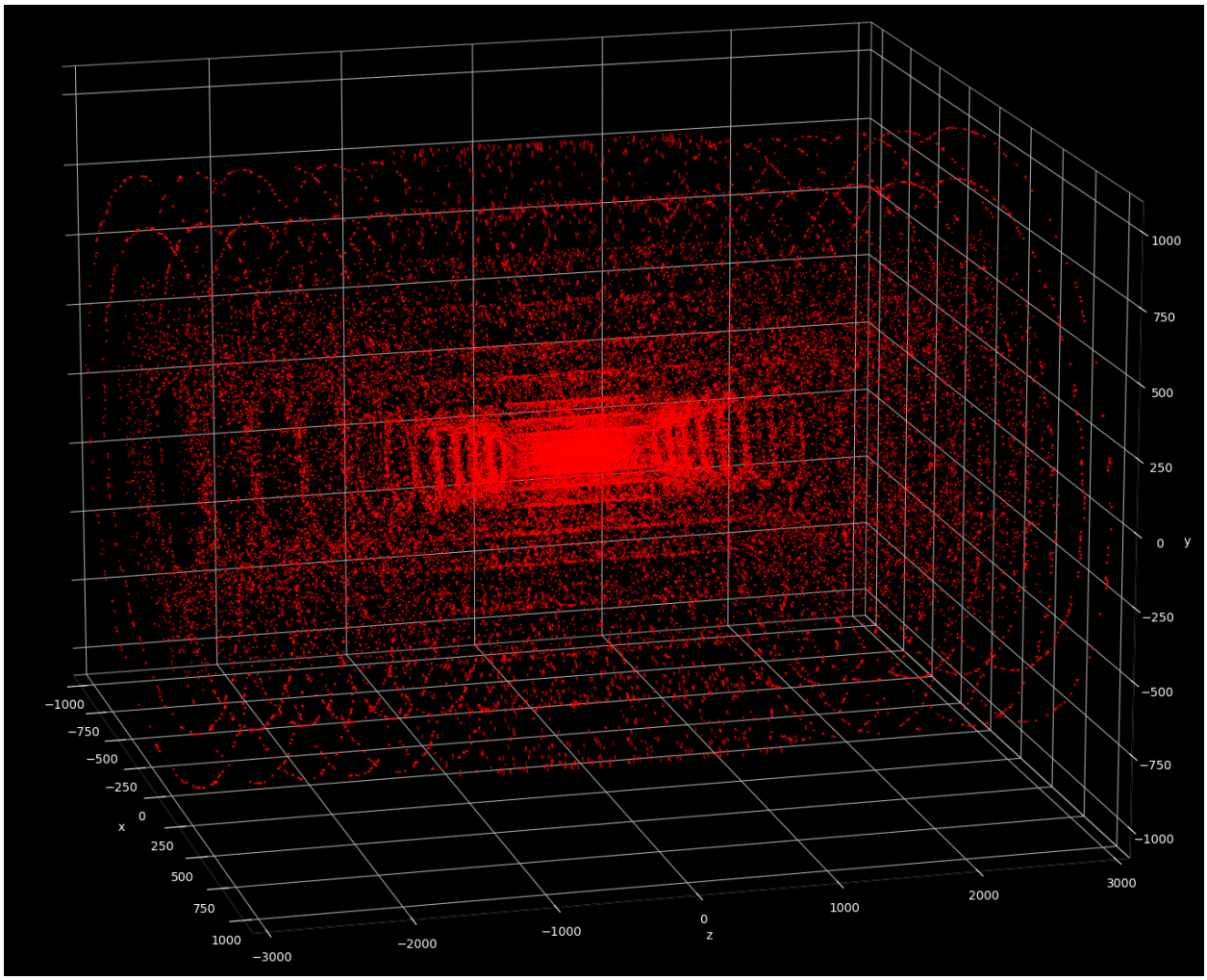


Figure 6: Measurements of a single event, 65279 spacepoints, $\mu = 200$. Note: different scales for x and r.

4 Adaptive Hough Transform

4.1 Hough Transform

The Hough Transform is one of the most popular methods for detecting simple shapes in images. It was originally devised for the detection of straight lines and patented in 1962 by Paul Hough [[Hou62]], and later generalized to detect other shapes. This technique remains widely used due to its simplicity, determinism, robustness to noise, occlusions, and gaps in the shape. The Hough Transform is based on the idea of representing shapes in the original space as points in the parameter space, with each point in the original space corresponding to a function in the parameter space. Hough space is discretized into cells and the functions intersecting with these cells are counted. The local maxima in the count are considered as shapes in the original space.

4.1.1 Hough Transform algorithm

The Hough Transform algorithm is illustrated using the example of detecting straight lines in an image consisting of multiple points. The first step is to find a way to represent lines from the original space as points in a parameter space and points of the original space as functions in the parameter space. The Hesse normal form can be used, as proposed by Richard O. Duda and Peter E. Hart [[DH72]].

A line l can be unambiguously represented as a point in the parameter space (θ, ρ), where θ is the angle between the x-axis and the line perpendicular to l. ρ is the distance from the origin to l. Figure 7 shows line l in the original and parameter spaces.

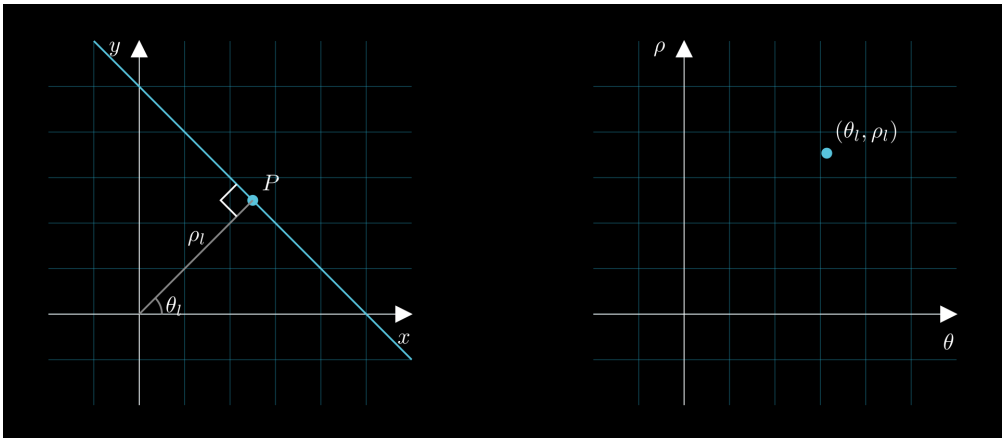


Figure 7: Line in original and Hough space.

A point P in the original space (x, y) can be represented as a function in the parameter space (θ, ρ) as follows (14):

$$\rho = x\cos(\theta) + y\sin(\theta) \tag{14}$$

As shown in Figure 8, collinear points in the original space have intersecting function representations in the Hough space. Applying the inverse transformation to that intersection results in the line in the original space that passes through all the collinear points.

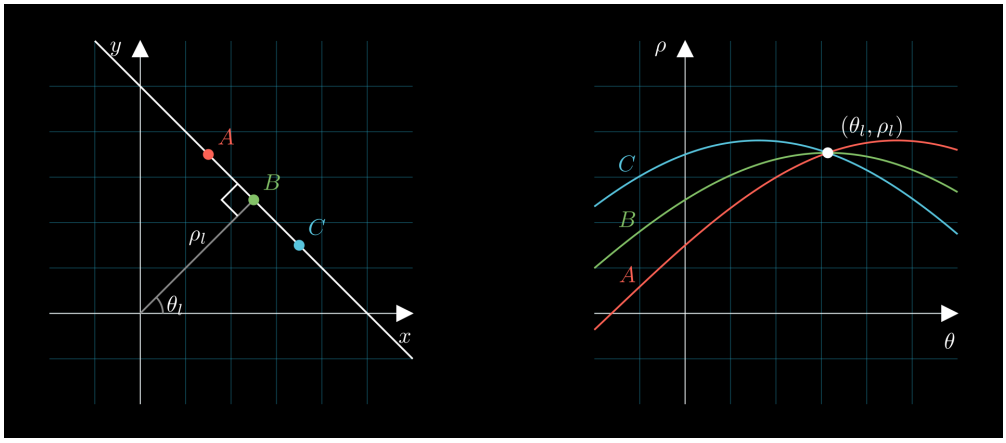


Figure 8: Collinear points in original and Hough spaces.

We can use that property to detect lines by quantizing the parameter space (θ, ρ) and counting the number of point functions passing through each cell. The quantized area of Hough space is referred to as the accumulator, and the process of filling it is known as voting or accumulator filling. Having filled the accumulator, we choose the cells with number of hits (votes) above a certain, arbitrary threshold. Each such cell is considered a line candidate with tolerance dependent on the quantization step. If necessary, other methods can be applied to the line candidates to improve the results. Depending on the chosen function representation, quantization granularity, and acceptance threshold, number of found solutions, including false positives, varies. Figure 9 illustrates the voting process.

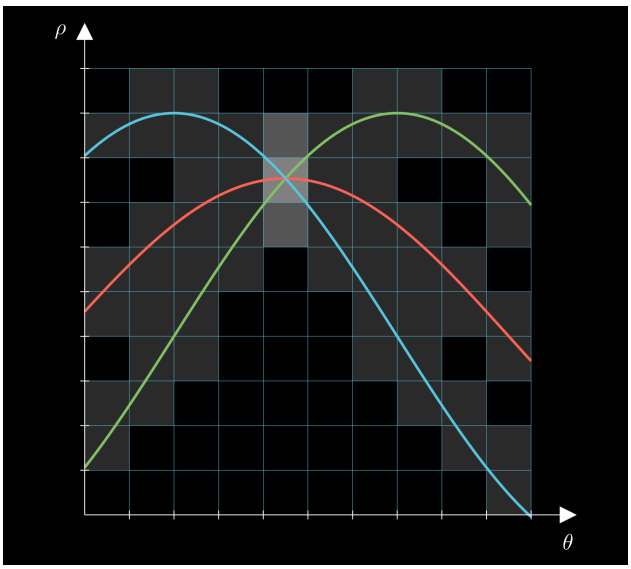


Figure 9: Hough accumulator filling example with 3 points.

4.1.2 Computational and memory complexity

In a naive implementation of the Hough Transform, we need to iterate over all points in the original space and, for each point, iterate over all cells in the Hough space. This results in $O(N \cdot C_\theta \cdot C_\rho)$ complexity, where N is the number of points in the original space, C_θ and C_ρ are the number of quantization steps in θ and ρ directions. However, the complexity can be improved by iterating over only one axis of the accumulator, for example θ, calculating the minimum and maximum values of ρ for the given θ range, and then voting for bins between them. This reduces the complexity to (15):

$$O(N \cdot C_\theta \cdot S_\rho) \tag{15}$$

where S_ρ is the average number of hits in the ρ direction for the chosen θ quantization step. S_ρ is highly dependent on the slopes of point representations in the ρ direction.

In both cases, the memory complexity is (16):

$$O(C_\theta \cdot C_\rho + N) \tag{16}$$

Hough Transform as used in this work utilizes a 2D accumulator, as will be described in the following sections. However, it is not possible to fully describe a helix using only two parameters. Therefore, helices have to be reduced to projections in 2D space, which results in loss of information and increases the number of false positives. It is possible to use a higher-dimensional accumulator, but it drastically increases computational complexity and memory requirements. The computational complexity for a D-dimensional accumulator is (17):

$$O(N \cdot C_1 \cdot S_2 \cdot S_3 \cdot \ldots \cdot S_D) \tag{17}$$

where:

- N - number of points in original space,
- C_1 - number of quantization steps in first dimension,
- S_i - average span of hits in i-th dimension for a (D-1)-dimensional slice.

The memory complexity for a D-dimensional accumulator is (18):

$$O(C_1 \cdot C_2 \cdot \ldots \cdot C_D + N) \tag{18}$$

where:

- N - number of points in original space,
- C_i - number of quantization steps in i-th dimension.

As can be deduced from the above formulas, time and memory complexities become prohibitive for higher dimensions or small quantization steps.

4.1.3 Hough space used in this work

As described in the trajectory characteristics subsection 3.1, particles we are interested in originate from interaction region and their trajectories bend in the plane perpendicular to the z-axis. For further considerations, it is assumed that every particle projected onto xy plane originates from interaction point $P(0, 0)$ and the radius of the helix is greater than the radius of the Inner Detector, resulting in trajectory being an arc spanning from P to the outer edge of the Inner Detector.

Following this projection of the trajectory onto the xy plane, it can be defined using two parameters - angle ϕ_t between x-axis and center of the arc (circle) and radius r_t of the arc. A projected spacepoint (measurement) S can also be defined using polar coordinates (r, ϕ). Figure 10 shows the trajectory and the spacepoint projected onto xy plane.

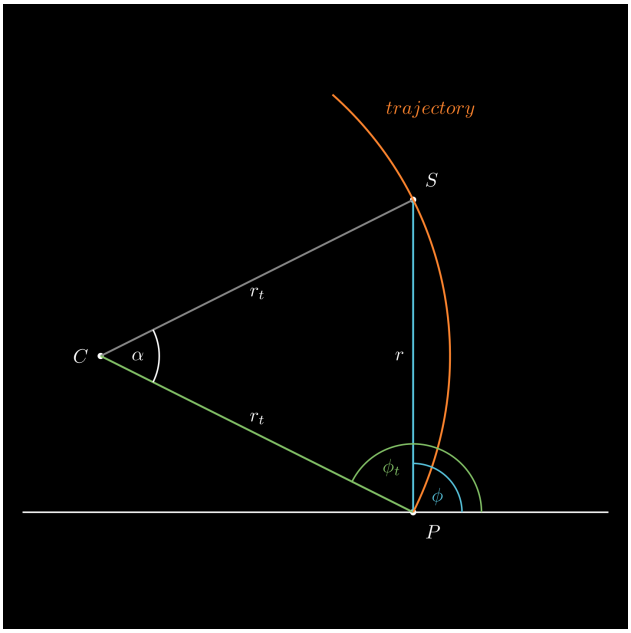


Figure 10: Trajectory and spacepoint in xy plane.

It is necessary to find a continuous, preferably bijective, function that maps r_t to ϕ_t (or vice versa) with r and ϕ as parameters. To achieve this, the tangent to the trajectory at point P is used as well as altitude of triangle CPS at point C, and a few more angles which are defined as follows:

- ϕ_0 - angle between x axis and tangent,
- δ - angle between tangent and section SP.
- γ - angle between CP and SP,

The tangent, altitude at C, and angles ϕ_0, δ, γ are depicted in Figure 11.

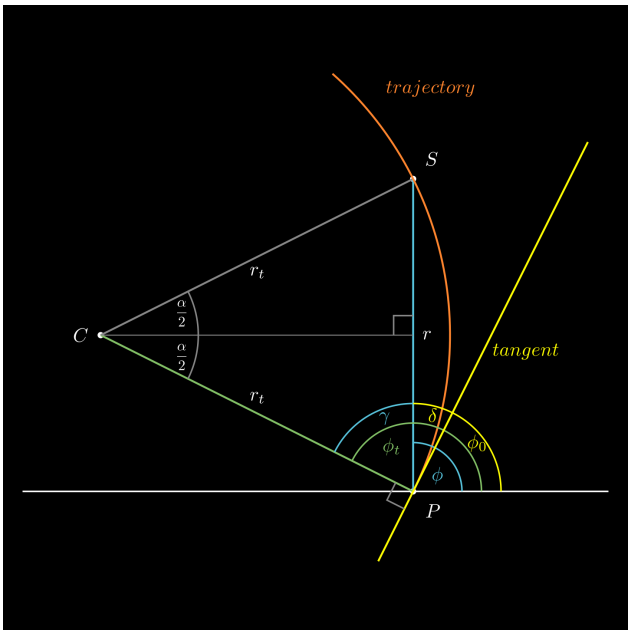


Figure 11: Trajectory and spacepoint in xy plane with tangent.

The following relationships can be concluded from Figure 11:

$$\frac{r}{2} = r_t \sin(\frac{\alpha}{2}) \tag{19}$$

$$\gamma = \pi - \frac{\pi}{2} - \frac{\alpha}{2} = \frac{\pi}{2} - \frac{\alpha}{2} \tag{20}$$

$$\delta = \frac{\pi}{2} - \gamma = \frac{\pi}{2} - (\frac{\pi}{2} - \frac{\alpha}{2}) = \frac{\alpha}{2} \tag{21}$$

$$\delta = \phi - \phi_0 \tag{22}$$

$$\phi_t = \phi_0 + \frac{\pi}{2} \tag{23}$$

Since only high momentum particles are of interest, small α can be assumed and $\sin(\frac{\alpha}{2})$ can be approximated as follows (24):

$$\lim_{\alpha \to 0^+} sin(\frac{\alpha}{2}) = \frac{\alpha}{2} \Rightarrow \sin(\frac{\alpha}{2}) \approx \frac{\alpha}{2} \tag{24}$$

Using the mentioned approximation and combining formulas (19), (22), (23), the following equations are derived:

$$\frac{r}{2} = r_t \sin(\frac{\alpha}{2}) \tag{25}$$

$$\frac{r}{2r_t} = \frac{\alpha}{2} \tag{26}$$

$$\frac{r}{2r_t} = \phi - \phi_0 \tag{27}$$

$$\phi_0 = \phi - \frac{r}{2r_t} \tag{28}$$

$$\phi_t = \phi - \frac{r}{2r_t} + \frac{\pi}{2} \tag{29}$$

Formula (29) is the desired bijective function mapping r_t to ϕ_t. Finally, it can be simplified by using ϕ_0 instead of ϕ_t as in formula (28) and account for charge of the particle by combining it with the formula (6) for radius of particle with momentum p_T derived in trajectory characteristics section 3.1.

$$\phi_0 = \phi - \frac{r}{2 \cdot \frac{p_T}{qB}} \tag{30}$$

$$\phi_0 = \phi - \frac{B}{2} \cdot \frac{q}{p_T} \cdot r \tag{31}$$

Hence, the above formula (31) for linear function of $\frac{q}{p_T}$ is obtained, which is used as Hough Transform in this project. $\frac{q}{p_T}$ and ϕ_0 are considered as trajectory parameters, implying $(\frac{q}{p_T}, \phi_0)$ accumulator space. ϕ and r are considered as $\phi_0(\frac{q}{p_T})$ function parameters. $\frac{B}{2}$ is a constant factor.

4.2 Adaptive Approach

The Adaptive Hough Transform (AHT) is a modification of the basic Hough Transform that lowers time and memory complexity for shape detection and improveds solution information retention without accuracy or efficiency loss. AHT detects shapes by progressively refining the accumulator through recursive subdivision, focusing exclusively on promising regions. The algorithm initiates with a single cell (1×1 accumulator resolution) covering the entire Hough space region of interest. Each cell is evaluated and subsequently either rejected or subdivided into multiple subcells. Each subcell undergoes the same evaluation process, and this procedure is repeated until the desired resolution is achieved. Cells that satisfy the evaluation criteria at the chosen division level are accepted as solutions. For each cell, voting is performed exclusively by points that intersect with the parent cell.

The step-by-step process of the Adaptive Hough Transform for a two-dimensional accumulator is as follows:

1. Determine the target accumulator resolution in each dimension and calculate the equivalent target division levels. The division level and resolution satisfy the relation $R_d = 2^{L_d}$, where R_d is the resolution in the d-th dimension and L_d is the division level in the d-th dimension.

2. Create a container for accumulator cells and an associative container mapping cells to sets of points.

3. Create a single accumulator cell covering the entire Hough space region of interest.

4. Assign 0 as the initial division level in every dimension for the cell.

5. Insert the cell into the cells container.

6. Create a map entry for the cell with a set containing all points to be processed by the algorithm.

7. Remove a cell from the cells container and evaluate it using the corresponding point set. In the most basic version, evaluation is equivalent to the Hough Transform voting process with a 1×1 accumulator covering the cell. However, evaluation can be extended with additional filters, as will be described in subsequent chapters.

8. If the cell does not meet the evaluation criteria, skip steps 9-13 and proceed to step 14.

9. If the cell's division level equals the target division level for all dimensions, accept it as a solution (add to the solutions set), skip steps 10-13, and proceed to step 14.

10. Divide the cell into subcells in dimensions where the cell's division level is less than the target division level. Subcells must cover exactly the same area as the parent cell and must not overlap. If they overlap, there may be duplicated solutions.

11. For every subcell created in step 10, assign appropriate division levels. Division levels for dimensions in which division occurred shall be one greater than the corresponding division levels of the parent cell. Division levels for other dimensions shall remain the same as those of the parent cell.

12. Insert the subcells into the cells container.

13. For each subcell created in step 10, create a cell-point set map entry containing the points that intersected with the parent cell in step 7.

14. If the cells container is not empty, return to step 7.

15. Return the solutions set.

Figure 12 illustrates the step-by-step filling of the accumulator. Each image shows the state of the accumulator after subsequent cycles of the algorithm in steps 7-14. Cells are colored according to the following scheme:

- **Green**: cell has been created as a division result in this cycle or accepted as a solution
- **Blue**: cell has not yet been evaluated
- **Red**: cell has been evaluated and rejected in this cycle
- **Gray**: cell has been evaluated and rejected in previous cycles; brightness is proportional to the number of hits in the cell

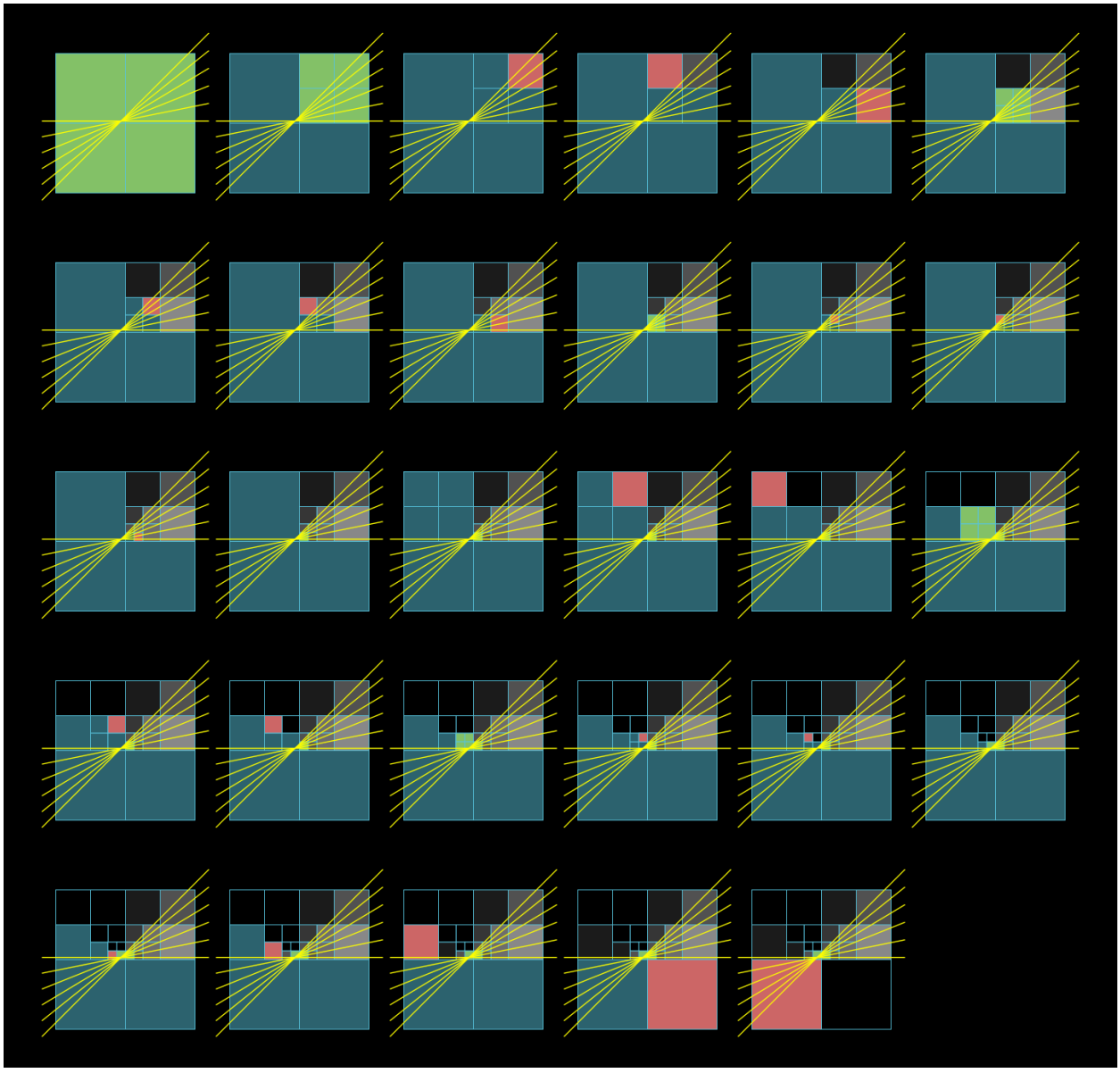


Figure 12: Filling of the accumulator in subsequent cycles of the algorithm.

Note that only cells colored green and blue need to be stored in memory.

If the evaluation criteria in step 7 constitute a simple voting process and cell division in step 10 involves halving in every affected dimension, the Adaptive Hough Transform yields identical results to the basic Hough Transform for the chosen resolution. Figure 13 illustrates the algorithm results for a 2D accumulator using both traditional and adaptive approaches. Cells colored green are accepted as solutions. Cells colored gray have been evaluated and rejected. The brightness of each cell is proportional to the number of hits within the cell. The number of cell evaluations for the traditional accumulator is 1024, while for the adaptive approach it is 37. Each cell in the traditional accumulator has been evaluated with 6 points. The average number of points used for evaluation in the adaptive approach is < 6.

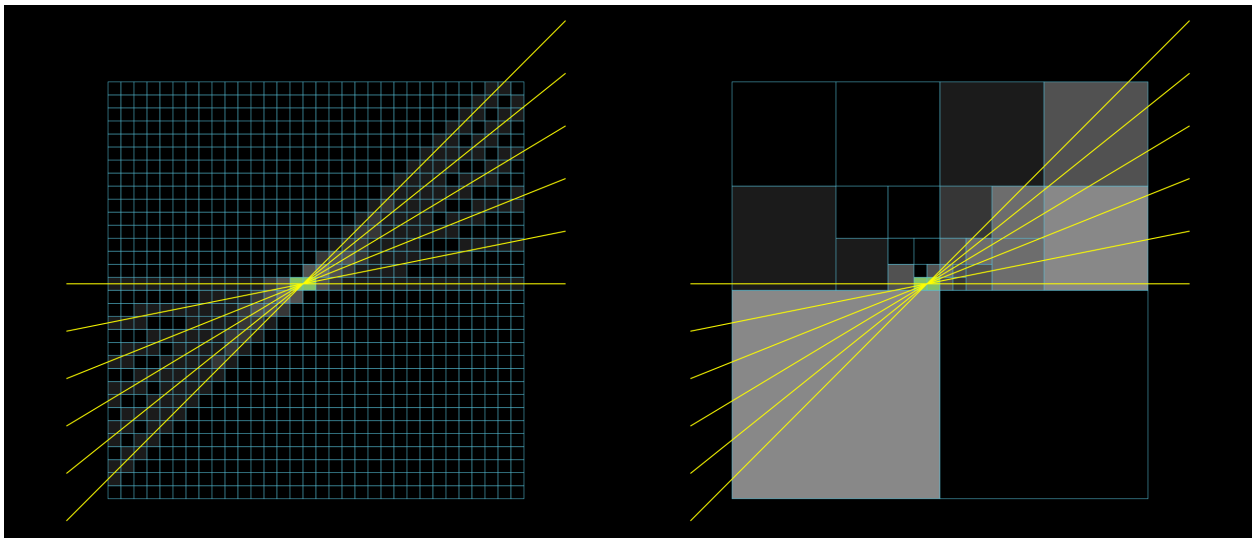


Figure 13: Traditional (left) and adaptive (right) accumulator filled with 6 points. Both accumulators have 32 cell resolution in each dimension and yield identical results.

4.3 Computational and Memory Complexity

While the classical Hough Transform utilizes a fixed grid covering the entire parameter space at the desired resolution and requires storage of the entire grid in memory, the Adaptive Hough Transform necessitates storage of only the cells that have not yet been evaluated and their associated point sets. The optimal processing order for cells is LIFO for reasons that will be explained in following sections. The maximum number of stored cells in the case of a 2D accumulator with equal target division levels for both dimensions is $N_c = 4L$, where L is the target division level. The number of point sets that must be stored is $N_p = N_c + 2$. Note that the set size for a subcell is always less than or equal to the size of the set associated with the parent cell, and on average, it decreases significantly with increasing division level. For higher dimensions, the maximum number of stored cells

is given by the following equation (32):

$$N_c = \sum_{i=1}^{L_{max}} 2^{D_i} \tag{32}$$

where:

- N_c - maximum number of stored cells,
- L_{max} - maximum division level in any dimension,
- D_i - number of dimensions for which division occurs at division level i.

Since AHT evaluates only regions of Hough space and points that demonstrate the likelihood of containing a solution, computational complexity is significantly lower than in the case of the classical Hough Transform. Computational complexity improves especially for sparse point distributions, high-dimensional accumulators, and high accumulator resolutions. The Big-O complexity for AHT is approximately the same as for classical HT, but this worst-case scenario is very unlikely. Θ for AHT is much lower than O whereas $\Theta \approx O$ for classical HT. The actual complexity cannot be precisely defined as it is highly dependent on the data distribution and algorithm parameters.

Due to the superior performance of AHT in high-dimensional accumulators, it enables the use of higher-dimensional parameter spaces, thereby achieving greater efficiency and fewer false positive solutions. A higher number of parameters also allows for the detection of lower-energy particles and particles not originating from the interaction region, as some assumptions for trajectory shape can be relaxed. Table 1 presents memory requirements as the peak number of points and cells (comparable memory footprint) that must be stored for different numbers of dimensions. Calculations assume target division level 11 for ϕ_0 and 9 for $\frac{q}{p_T}$ for the 2D accumulator, as these parameters proved to be near-optimal in tests that will be discussed later. For higher dimension cases, a target division level is 9 for every dimension. The number of points for all cases is 1000. For AHT, the size of the point set is unrealistically pessimistic and assumes it is always equal to the number of points. The number of 5 dimensions is significant, as in case of problem discussed in this work, a helix can be fully described by 5 parameters.

Dimensions	HT memory requirement (cells)	AHT memory requirement (points)
2	$2^{9+11} \approx 5 \cdot 10^6$	$10^3(2^2 \cdot 9 + 2 \cdot 2) = 5 \cdot 10^4$
3	$2^{9 \cdot 3} \approx 1.3 \cdot 10^8$	$10^3(2^3 \cdot 9) \approx 7.2 \cdot 10^4$
4	$2^{9 \cdot 4} \approx 6.9 \cdot 10^{10}$	$10^3(2^4 \cdot 9) \approx 1.4 \cdot 10^5$
5	$2^{8 \cdot 5} \approx 3.5 \cdot 10^{13}$	$10^3(2^5 \cdot 9) \approx 2.9 \cdot 10^5$

Table 1: Memory requirements for Hough Transform and Adaptive Hough Transform

From a computational complexity perspective, a major advantage of AHT is its handling of irrelevant points, such as spacepoints not originating from high-p_T particles or not intersecting the cells at all. In the classical HT, all points vote across the entire accumulator, incurring the full computational cost even for non-intersecting points. AHT discards such points early in the process. This advantage is particularly significant due to the concentration of points near the interaction region and the division of detector space, which will be described later.

4.4 Other Characteristics

4.4.1 Adaptive Resolution

AHT allows for adaptive resolution based on a number of factors that may include:

- number of points in the cell
- local density of points
- statistical analysis of points in the cell
- placement of the evaluated region in the parameter space

4.4.2 Solution Storage

Since AHT traverses the accumulator in a depth-first manner, solutions can be streamed out to other components of the data processing pipeline as they are found, instead of storing them in memory.

4.4.3 Branching

AHT relies on frequent conditional instructions to determine the next steps. Frequent branching reduces the benefits of branch prediction and SIMD vectorization. The impact is more severe on GPUs.

4.4.4 Cache Efficiency

Using a stack-based implementation of AHT allows for better utilization of processor cache, since points and cells queued for processing in LIFO order exhibit high temporal locality and hence have a high probability of being in cache. The effect should be especially noticeable at high division levels, when point sets are small.

4.4.5 Low Cost of Resolution Increase

The cost of increasing resolution is low due to the smaller sizes of point sets at higher division levels and the aforementioned data locality.

4.4.6 Solution Information Retention

AHT naturally preserves more information about found solutions compared to the standard Hough Transform. Since solutions are synonymous with cells at the end of processing, they can be associated with a complete list of points that contributed to the solution. This information can be used to further evaluate the solution.

4.4.7 Additional Filtering

As the number of points potentially contributing to a solution decreases with division level and a complete list of such points is available, additional filtering and optimizations can be applied at the cell evaluation step. Such filters can be easily included in or excluded from the algorithm. Possible filters include:

- relative point distribution
- point values in dimensions not contributing to representation in Hough space
- point distribution uniformity
- analytical filters such as crossing in Hough space
- filters based on domain-specific constraints

Two such filters were implemented, as will be described in later chapters.

5 The Algorithm

This chapter describes in detail the final trajectory finding algorithm used in this project. The description is divided into three parts. The first part describes the splitter - a component responsible for spatial partitioning of the measurements into batches. Each batch is processed as a separate run of AHT in a component named the helix solver, described in the second part. The third part describes filters used at the AHT cell evaluation step in addition to hit counting.

5.1 Terms and Definitions

Further descriptions follow a consistent naming and measurement convention for angles, directions, and parameters, the same as used in the implementation. The convention is as follows:

- x_{angle}: angle in the plane perpendicular to the x axis (zy plane), measured counterclockwise from positive half of the z axis
- z_{angle}: angle in the plane perpendicular to the z axis (xy plane), measured counterclockwise from positive half of the x axis
- x_{angle} range or x_{range}: range of x-angles
- z_{angle} range or z_{range}: range of z-angles
- interaction region: a line segment on the z axis, spanning between real interaction region boundaries in the z direction

5.2 Splitter

Particles created in the interaction region spread away from the interaction region, moving on helical trajectories with high momentum. As a result, traces left by a given particle are always located in a narrow region of the detector space. This fact can be used to reduce the number of points that must be processed together for a given Hough space region by splitting the detector space into regions processed separately. The component responsible for this is called the splitter.

The splitter divides the detector space into regions of two types: wedge-shaped regions covering most of the volume and cone-shaped regions covering volumes stretching from the interaction region ends to the outer edges of the detector along and close to the z-axis.

5.2.1 Wedges

A wedge is a region of space spanning from the interaction region to infinity, enclosed on four sides by planes defined by the following five parameters:

- z_{angle} minimum
- z_{angle} maximum
- x_{angle} minimum
- x_{angle} maximum
- interaction region width

Figure 14 shows 8 overlapping wedges around the z-axis colored in red and yellow. Note that the fifth plane, limiting distance from the z-axis, is drawn only for visualization purposes.

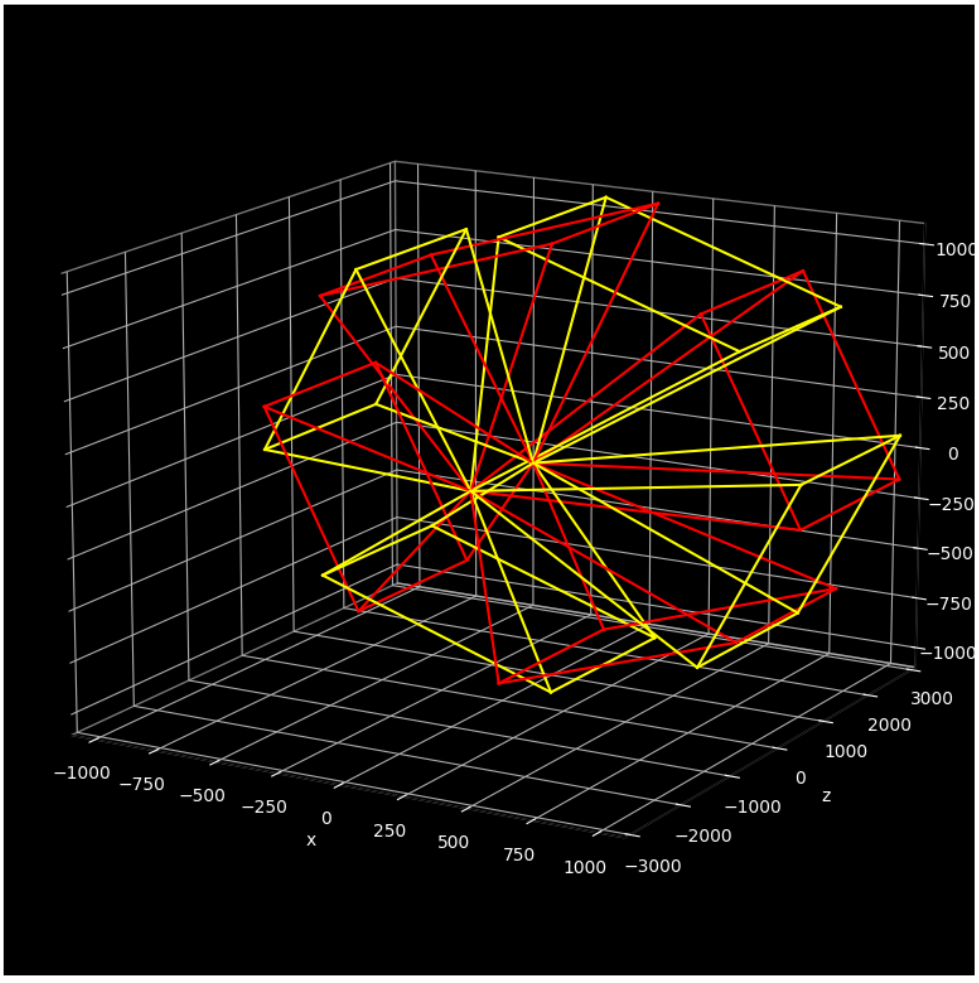


Figure 14: 8 overlapping wedges around z-axis colored in red and yellow.

Since the interaction point and particle movement direction are not correlated, a particle created near the low z end of the interaction region may move at a low z_{angle} as well as at a high z_{angle}. For successful trajectory reconstruction, all traces left by a particle must be included together in at least one wedge. Thus, every wedge covers the entire interaction region, as can be seen in Figure 15, which shows several wedges from a direction parallel to the x-axis. Unfortunately, this results in a high number of spacepoints being associated with multiple wedges and a higher processing load.

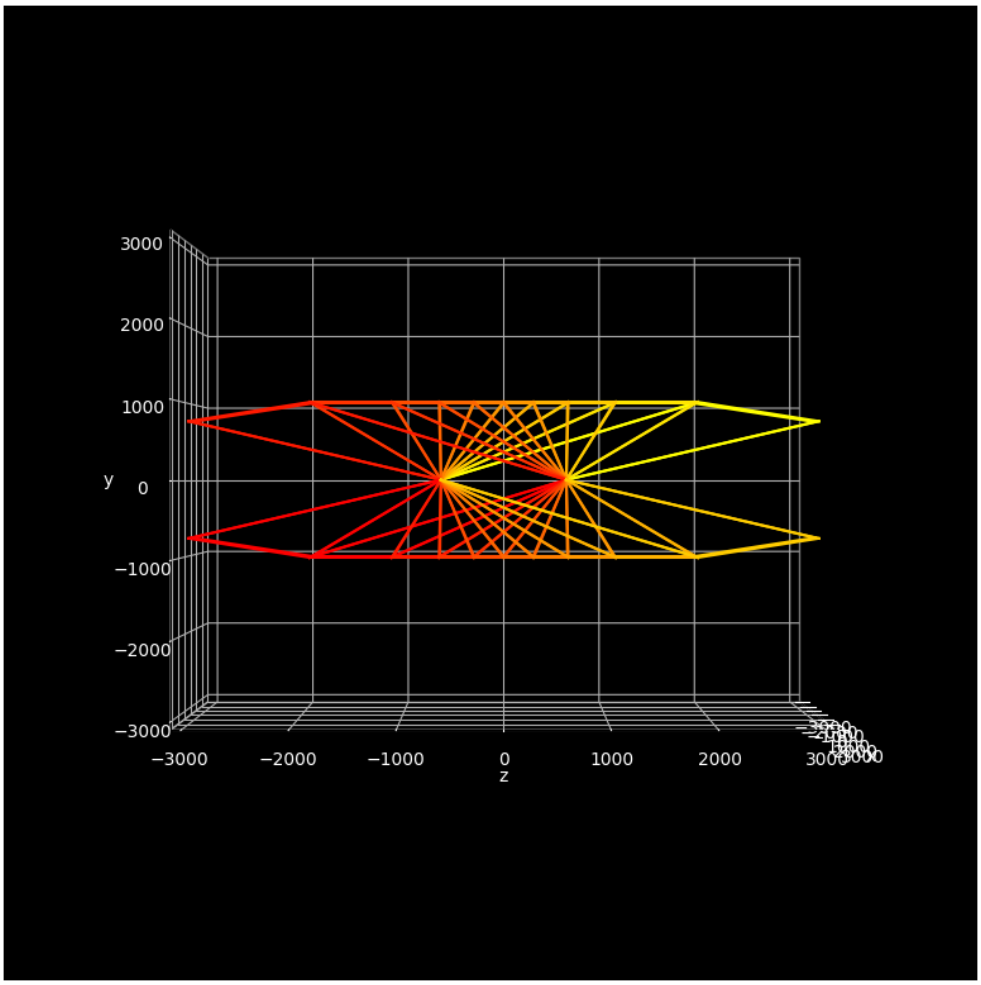


Figure 15: Several wedges from a direction parallel to the x-axis.

The implementation imposes the following restrictions on wedge parameters:

- $z_{angle_max}, z_{angle_min} \in [0, 2\pi)$
- $|z_{angle_max} - z_{angle_min}| < \pi$
- $(z_{angle_max} - z_{angle_min} < \pi \wedge z_{angle_max} > z_{angle_min}) \vee (z_{angle_max} + 2\pi - z_{angle_min} < \pi \wedge z_{angle_max} < z_{angle_min})$
- $x_{angle_max}, x_{angle_min} \in (0, \pi)$
- $x_{angle_max} > x_{angle_min}$

5.2.2 Pole Regions

Pole regions are two conical regions with cone axes parallel to the z-axis. The splitter associates points with the pole regions, but they are not used by the helix solver, as traces left exclusively in these regions belong to high x_{angle} (high η) particles, which are not targeted by our solution, as mentioned in the assumptions subsection 2.2. Pole regions are visualized in Figure 16. Pole regions are defined by two parameters:

- x_{angle}: angle between the z-axis and cone side
- interaction region width

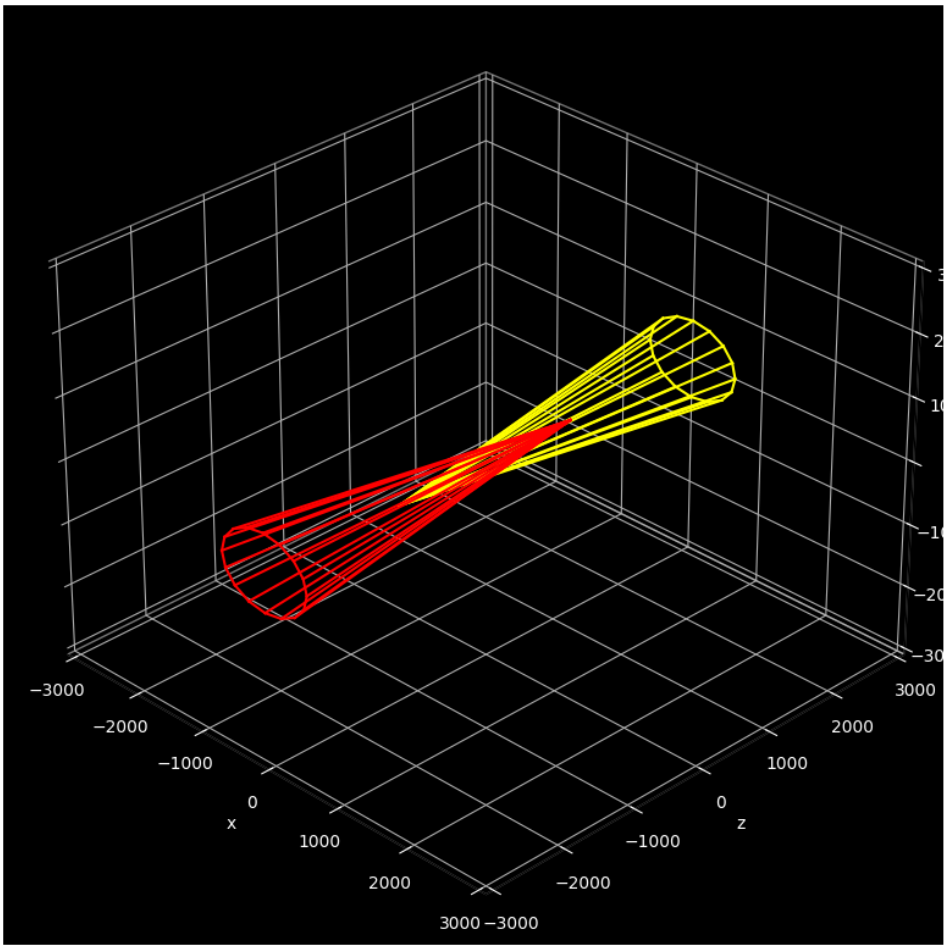


Figure 16: Pole regions.

The implementation imposes the following restrictions on pole region parameters:

- $x_{angle} \in (0, \frac{\pi}{2})$ for regions lying mostly in $z > 0$
- $x_{angle} \in (\frac{\pi}{2}, 2\pi)$ for regions lying mostly in $z < 0$

5.2.3 Margins

Some particles may have trajectories such that the traces left by them lie on a regions boundary. If neighboring regions were not overlapping, such particles would be lost, as some of the traces would be associated with one region and some with the other. To avoid losing particles, all regions used in the splitter are overlapping by some margin, as visualized in Figure 14. Margins are distinct for x and z angles and are added to the region parameters. In subsequent chapters, margins are measured as the difference in angle of a limiting plane before and after adding the margin. In other words, for wedges, the margin angular width is half of the angular width of the overlapping space.

5.2.4 Splitter Regions Used for Most Spacepoints Full Event and Performance Tests

Figure 17 visualizes all the wedges and pole regions generated with the settings listed below. This configuration has half the number of regions in the x and z directions compared to the configuration used for most spacepoints full event and performance tests discussed in later chapters. Additionally, margins are set to 0 to make the image clearer. Visualizing all regions with margins used in the mentioned tests would result in a red and blue blob due to the number of lines.

Splitter settings:

- number of x_{angle} ranges: 24
- number of z_{angle} ranges: 16
- total x_{angle} span: $[\frac{5}{256}\pi, \frac{251}{256}\pi]$
- total z_{angle} span: $[0, 2\pi)$
- pole region x_{angle}: $\frac{5}{256}\pi$
- x_{angle} margin: 0
- z_{angle} margin: 0
- interaction region width: 400 mm

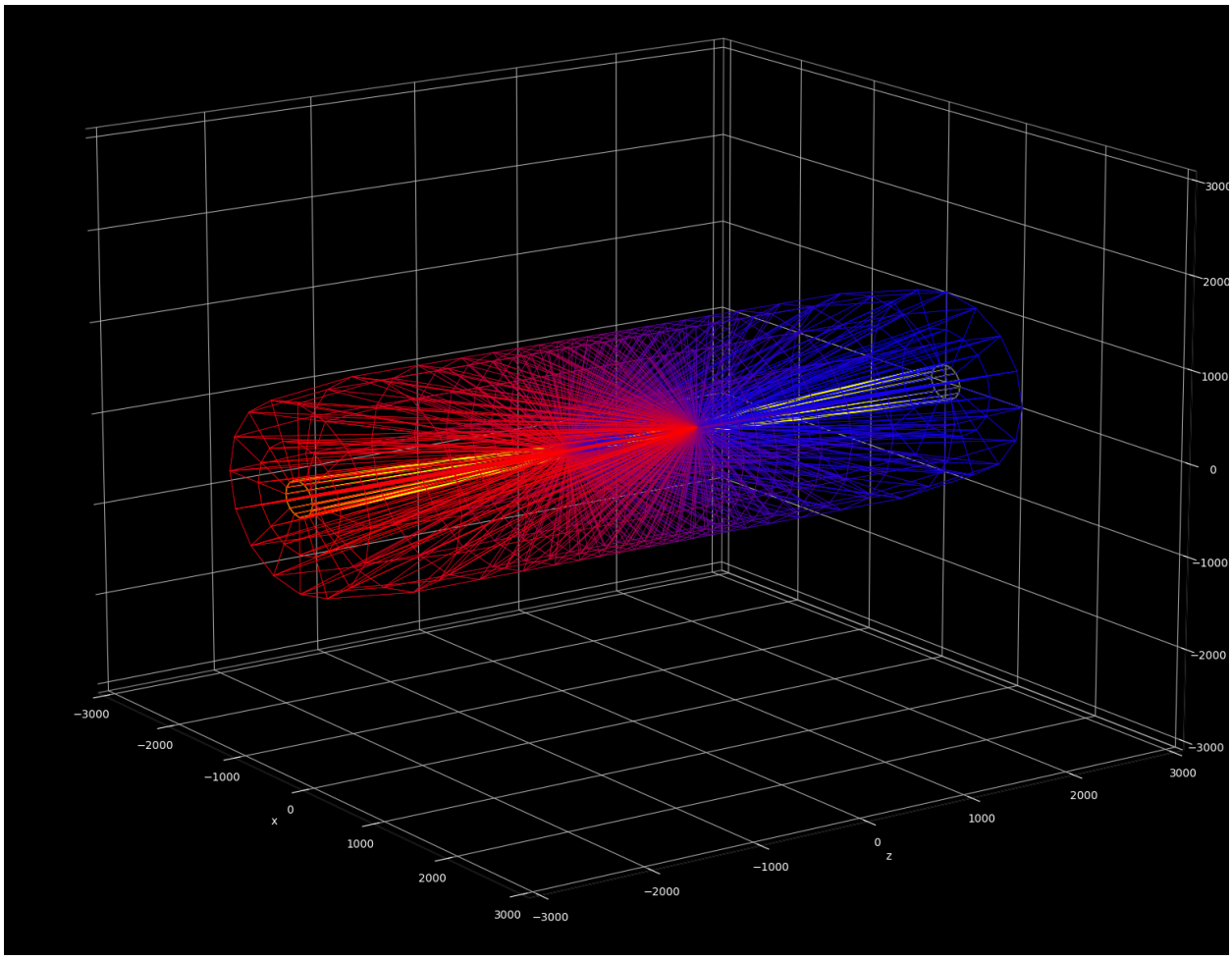


Figure 17: Splitter regions visualization. Pole regions are colored in yellow, wedges are colored with a blue-to-red gradient based on x_{angle}.

5.2.5 Innermost Layer Rejection

In addition to splitting into regions, the splitter also filters out points located in a cylindrical fragment of space around the interaction region. The rejection region is parametrized, and the used parameters are mentioned in the helix solver parameters subsubsection 5.3.1. Points in this region are rejected to reduce the number of false positives and decrease the processing load. These points are problematic for three main reasons:

- Traces in inner layers are left mostly by low-energy particles, which are noise from our perspective.
- The density of points in the innermost layers is so high that it is not possible to conclusively associate a given point with a single particle (or just a few). This results in solutions consisting in large proportions of inner layer points. Hence, a trade-off must be made, by adjusting the hits threshold, between the number of false positives and the number of missed particles.

- The splitter associates points close to the interaction region with many regions, vastly increasing the processing load for the helix solver.

5.3 Helix Solver

`HelixSolver` is a component wrapping the entire trajectory finding process. `HelixSolver` receives `Task` objects containing an `Event` and a `Result`. `Event` holds spacepoints and event meta-information. `Result` is a container for solutions. Three main steps of solving a `Task` are memory allocation, splitter run, and AHT run for each region. Each AHT run consists of the following steps:

1. **Parameter setting**: AHT parameters are region-dependent and selected based on settings that will be described later.

2. **Point coordinates conversion**: points are converted to polar coordinates and the distance unit is converted from millimeters to meters.

3. **Layer assignment**: each point is assigned a layer number used for additional filters described in the layer hit check subsubsection 5.4.1 and line crossings check subsubsection 5.4.2.

4. **Region and points rotation**: due to `atan2` discontinuity, helixes crossing $y = 0$ on the negative x side cannot be correctly detected without special case handling. To avoid this, if the solved region contains $y = 0$ on the negative x side, the region and points are rotated by $-\pi$ around the z-axis.

5. **Initial accumulator cell creation**: the initial accumulator cell covers the entire Hough space region of interest. The cell is put on the cell stack.

6. **Initial cell point list filling**: a point list associated with the initial cell is filled with points that intersect the cell.

7. **AHT cell processing**: cells are processed until the cell stack is empty.

8. **Solutions rotation**: if the region was rotated in step 4, solutions are rotated back by π around the z-axis.

Cell processing proceeds as follows:

1. A cell is taken off the cell stack.

2. The cell is evaluated by checking the number of hits, number of layer hits (described in the layer hit check subsubsection 5.4.1), and number of line crossings (described in the line crossings subsubsection 5.4.2). If it fails any check, its processing ends

and the cell is discarded. This is also the step that could be altered to add or remove additional filters.

3. If the division level of the cell has not reached the target division level in all dimensions, the cell is divided into subcells.

4. For each subcell, a point list is created and filled with points that intersect the subcell. The subcell is put on the cell stack.

5. If the target division level was reached in all dimensions, a final check of the number of layer hits is made. If the cell has enough layer hits, it is accepted as a solution and placed in the `Result` container. Solutions are stored as q, r, ϕ_0, x_{angle_min}, x_{angle_max} values calculated from cell and wedge parameters. This step can be modified to add or remove additional information regarding solutions.

5.3.1 Helix Solver Parameters

`HelixSolver` is initialized with a set of parameters included in `SplitterSettings` and `Wedge` objects representing wedges. Additionally, some parameters are hardcoded in the `HelixSolver`, `RegionSolverData`, and `SplitterSettings` classes.

`SplitterSettings` parameters:

- `maxAbsXy_` [mm]: maximum absolute value of x and y coordinates of spacepoints. Should fit Inner Detector geometry (radius) with a small margin.

- `maxAbsZ_` [mm]: maximum absolute value of z coordinate of spacepoints. Should fit Inner Detector geometry (length) with a small margin.

- `minZAngle_` [rad]: minimum z_{angle} of area covered by wedges. By default, set to 0 to cover the entire xy plane.

- `maxZAngle_` [rad]: maximum z_{angle} of area covered by wedges. By default, set to 2π to cover the entire xy plane.

- `minXAngle_` [rad]: minimum x_{angle} of area covered by wedges.

- `maxXAngle_` [rad]: maximum x_{angle} of area covered by wedges.

- `poleRegionAngle_` [rad]: angle of pole regions. Should equal `minXAngle_` to cover all spacepoints.

- `interactionRegionMin_` [mm]: minimum z coordinate of interaction region.

- `interactionRegionMax_` [mm]: maximum z coordinate of interaction region.

- `zAngleMargin_` [rad]: margin added to z_{angle} of wedges.
- `xAngleMargin_` [rad]: margin added to x_{angle} of wedges.
- `numZRanges_`: number of equal angular width splits into wedges of space defined by `minZAngle_` and `maxZAngle_`.
- `numXRanges_`: number of equal angular width splits into wedges of space defined by `minXAngle_` and `maxXAngle_`.
- `filterOutCenterR_` [mm]: radius of cylindrical space around interaction region that is filtered out. See subsubsection 5.2.5.
- `filterOutCenterZ_` [mm]: distance from interaction region center to the edge of the filtered out region in z direction. See subsubsection 5.2.5.

`Wedge` parameters:

- `solutionHitsThreshold_`: hits threshold of AHT used for this wedge.
- `linesCrossingsThreshold_`: line crossing threshold of AHT used for this wedge. See subsubsection 5.4.2.
- `skipCrossingsCheckThreshold_`: line crossing check is skipped if the number of hits in the cell is greater than this threshold.

Hardcoded `SplitterSettings` parameters:

- `MaxWedgesNum`: maximum number of wedges. Used for memory allocation. The optimal value is `numXRanges_` $\cdot$ `numZRanges_`.

Hardcoded `HelixSolver` parameters:

- `SpaceMaxPhiPhi0AbsDiff` [rad]: maximum absolute difference between ϕ and ϕ_0. This value is used to determine the ϕ_0 range of the Hough space region. The default value is $\frac{\pi}{4}$.
- `SpaceMinQOverPt`: minimum q/p_t of Hough space region. The default value is -0.31 and is equivalent to a particle with p_t slightly lower than $1GeV/c$ and negative charge.
- `SpaceMaxQOverPt`: maximum q/p_t of Hough space region. The default value is 0.31 and is equivalent to a particle with p_t slightly lower than $1GeV/c$ and positive charge.

- `SolutionHitsThreshold`: hits threshold of AHT. Used only if `solutionHitsThreshold_` is 0 for the processed wedge.

- `LinesCrossingsThreshold`: line crossing threshold of AHT. Used only if `linesCrossingsThresh` is 0 for the processed wedge.

- `SkipCrossingsCheckThreshold`: line crossing check skip threshold. Used only if `skipCrossingsCheckThreshold_` is 0 for the processed wedge.

- `BMagnitude`: magnetic field magnitude. The default value is $2.0T$.

Hardcoded `RegionSolverData` parameters:

- `MaxPointsInRegion`: maximum number of points in a region. Should be tuned to splitter settings and be as low as possible but no lower than the number of points in the largest region.

- `Phi0MaxDivisionLevel`: maximum division level for ϕ_0.

- `QOverPtMaxDivisionLevel`: maximum division level for q/p_t.

- `LayerHitThreshold`: threshold for the number of layer hits. See the layer hit check subsubsection 5.4.1.

- `MaxPointListsPointsNum`: maximum number of points stored at the same time in all point lists combined. Should be as low as possible. By default, it is pessimistically set to `MaxPointsInRegion` $\cdot$ `MaxPointListsNum`.

Other hardcoded `RegionSolverData` parameters; optimal values are calculated automatically at compile time:

- `MaxDivisionLevel`: maximum division level for any dimension, $\max($ `Phi0MaxDivisionLevel`, `QOverPtMaxDivisionLevel` $)$.

- `MaxAccumulatorRegionStackSize`: maximum number of accumulator regions that can be stored on the stack, `MaxDivisionLevel` $\cdot 4$.

- `MaxPointListsNum`: maximum number of point lists that can be stored at the same time, `MaxDivisionLevel` $+ 2$.

5.4 Additional AHT Filters

5.4.1 Layer Hit Check

Spacepoints located in close proximity tend to be classified by basic hit counting as creating the same helix. This effect is especially pronounced in the inner layers of the detector where the density of spacepoints is high. Due to trajectory shape and detector geometry, it is not possible for a high-energy particle to leave more than one trace in the same layer. Thus, the hit count is inflated by coincidentally close spacepoints. However, the fact that the detector consists of layers can be used to mitigate this effect using a layer hit check. This filter discards cells that have hits belonging only to a number of layers lower than some threshold.

Every point is assigned a layer number based on its location in the rz plane projection. Criteria for layer assignment were chosen based on the data sets used in tests and may differ from the actual detector geometry. There are 23 layers divided into 5 groups with criteria for r and z specified in the Table 2 and visualized in Figure 18.

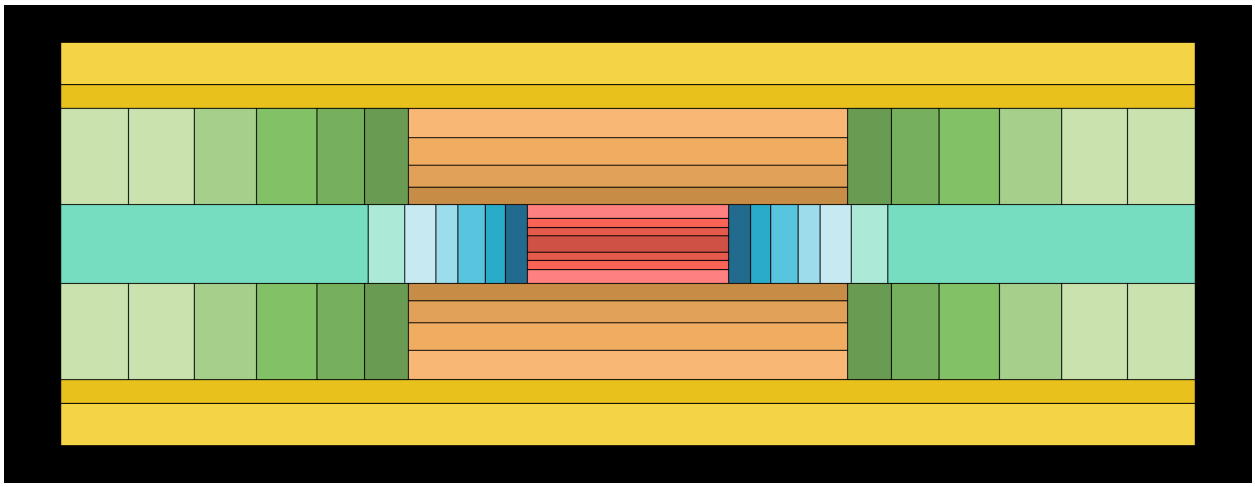


Figure 18: Detector layers in rz plane projection with color-coded groups.

Layer number	Layer group	Min r [mm]	Max r [mm]	Min $\|z\|$ [mm]	Max $\|z\|$ [mm]
1	A	0	45	0	550
2	A	45	90	0	550
3	A	90	140	0	550
4	A	140	215	0	550
5	B	0	215	550	670
6	B	0	215	670	780
7	B	0	215	780	930
8	B	0	215	930	1050
9	B	0	215	1050	1220
10	B	0	215	1220	1420
11	B	0	215	1420	∞
12	C	215	310	0	1200
13	C	310	430	0	1200
14	C	430	580	0	1200
15	C	580	740	0	1200
16	D	740	870	0	∞
17	D	870	∞	0	∞
18	E	215	740	1200	1440
19	E	215	740	1440	1700
20	E	215	740	1700	2030
21	E	215	740	2030	2370
22	E	215	740	2370	2730
23	E	215	740	2730	∞

Table 2: Detector layers.

5.4.2 Line Crossing Check

As can be seen in Figure 19, spacepoints that are nearly parallel to each other pass the basic hit count check in multiple cells and result in multiple false positive solutions (no crossing within the cell) or subdivisions of regions that do not contain any true positive solutions. An obvious solution would be to add a filter that discards cells that do not have a sufficient number of line crossings. However, such a filter comes with two drawbacks:

- The time complexity of checking the number of lines crossing within a cell for a set of N lines is $O(N^2)$, which is non-negligible for our application.
- To leave a trace, a particle must interact with the detector and therefore exchange energy with it. This means that each measurement deflects particle's trajectory or

changes its momentum. As a result, traces of a particle in Hough space do not all cross perfectly at a single point. Instead, they form a cloud of intersections concentrated near the true solution, as visualized in Figure 20.

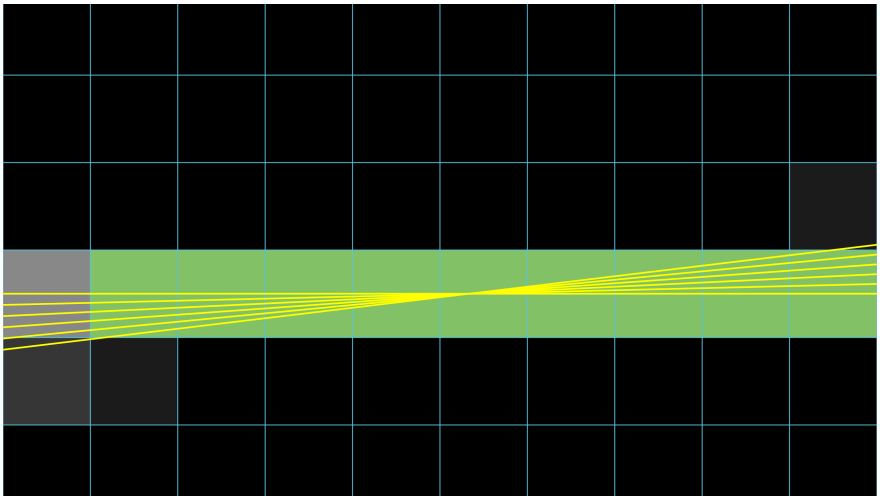


Figure 19: Accumulator with points passing the hit check for multiple cells but crossing in only one.

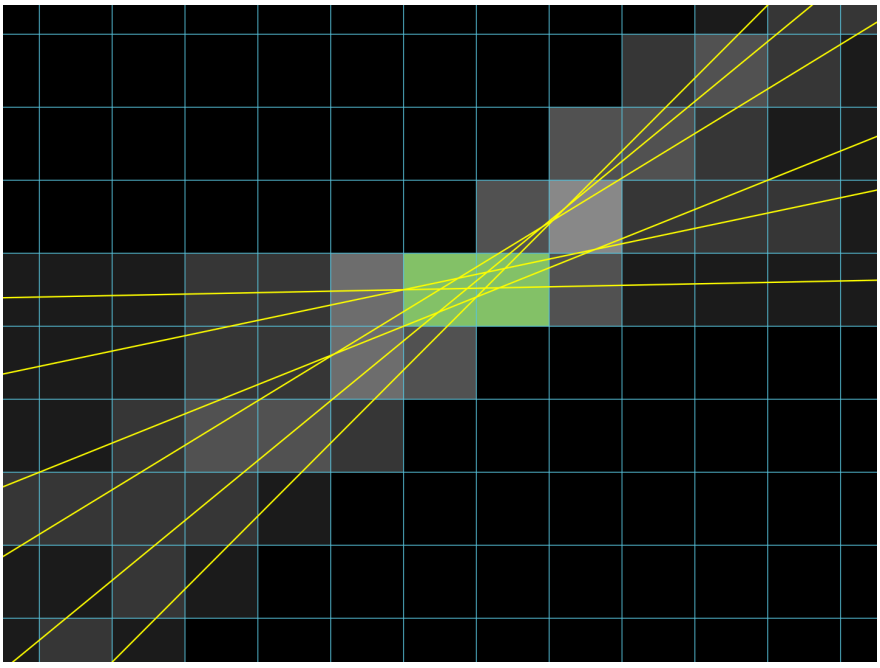


Figure 20: Cloud of intersecting lines near true solution.

The time complexity problem is mitigated by the introduction of a hits threshold above which the line crossing check is skipped. Since the check is applied later, the value of the threshold does not impact final result. Hence, an optimal threshold value for a given set of solver parameters exists.

The problem of imperfect measurements is mitigated by extending the area in which line crossings are counted at high division levels. In the implementation of the algorithm

used in this project, the area is extended for division levels higher than 6 to the size of a cell at division level 6.

Since lines are monotonic and continuous functions, crossing within a given argument span can be determined by evaluating the endpoints. If the relative ordering on the ϕ_0 axis reverses, a crossing is guaranteed to be present in the $\frac{q}{p_t}$ interval between the endpoints. Specifically, if

$$(\phi_{0A}((\frac{q}{p_t})_{min}) > \phi_{0B}((\frac{q}{p_t})_{min})) \wedge (\phi_{0A}((\frac{q}{p_t})_{max}) < \phi_{0B}((\frac{q}{p_t})_{max})) \tag{33}$$

or

$$(\phi_{0A}((\frac{q}{p_t})_{min}) < \phi_{0B}((\frac{q}{p_t})_{min})) \wedge (\phi_{0A}((\frac{q}{p_t})_{max}) > \phi_{0B}((\frac{q}{p_t})_{max})) \tag{34}$$

then a crossing is guaranteed to be present in the $\frac{q}{p_t}$ interval between $(\frac{q}{p_t})_{min} \wedge (\frac{q}{p_t})_{max}$.

Additionally, for a line crossing to count, the intersecting lines must come from different layers. Effectively, this implies that a cell passing line crossing check also passes the layer hit check with the same threshold.

6 Solution Quality Tests

Multiple tests were performed to evaluate the quality of solutions found by the solver configured with different parameter sets. The tests can be grouped into three main categories based on the type of a test and the data used. The first group, named "Run Per Particle Initial", runs the solver on a set consisting of traces left by one particle with a near-perfect trajectory. The second group, named "Particles Initial", utilizes a set of spacepoints generated in the same way, but all spacepoints are processed in a single helix solver run. The third group, the "Spacepoints" tests, use sets of spacepoints described in the data structure and characteristics section 3. Each test runs on a full simulated measurements set and represents the closest scenario to a real use case.

Run Per Particle Initial (RPPI) and Particles Initial (PI) tests use data generated based on the position components of particles initial described in section 3.3 that meet the $r < 0.1$ mm, $|z| < 200$ mm criteria for origin location and have pseudo-randomly chosen trajectory radius and charge.

Binomial proportion confidence intervals for all tests are calculated using the Wilson Score method [Wil27].

6.1 Run Per Particle Initial

Each set of spacepoints associated with a single particle is processed in a separate helix solver run, which excludes any interference between neighboring particles. Spacepoints were obtained using a simple generator that distributes a chosen number of points uniformly in the r dimension on a helix shape of given parameters. Radii were chosen randomly from the range $[1666, 20000]$ mm. Finding helixes with greater radii should not pose a problem, as they better fit the assumptions of their radius being much greater than the Inner Detector radius. This kind of test is suitable for evaluating the solver's ability to correctly reconstruct trajectories in idealistic conditions and to find the solver's response to parameter changes.

For every RPPI test, a particle trajectory is considered found if, for the corresponding helix solver run, at least one solution meeting the following criteria is found:

- $r \in (0.95 \cdot r_{true}, 1.05 \cdot r_{true})$
- $q = q_{true}$
- $z_{angle} \in (z_{angle_true} - \frac{\pi}{64}, z_{angle_true} + \frac{\pi}{64})$
- $x_{angle_min} < x_{angle_true} + \frac{\pi}{64}$
- $x_{angle_max} > x_{angle_true} - \frac{\pi}{64}$

6.1.1 Efficiency by x- and z-angle

Figure 21 shows the efficiency for 64 equal-width bins, each containing particles with x_{angle} in the range of the bin. Note that the bins overlap by double x_{angle} margin and correspond to wedges of the splitter. The solver found solutions for all 4633 particles, proving its ability to find helixes in a wide pseudorapidity range.

The test was run with the following settings:

- `minZAngle_`: 0
- `maxZAngle_`: 2π
- `minXAngle_`: $\frac{5}{256}\pi$
- `maxXAngle_`: $\frac{251}{256}\pi$
- `poleRegionAngle_`: $\frac{5}{256}\pi$
- `zAngleMargin_`: $\frac{1}{32}\pi$
- `xAngleMargin_`: $\frac{1}{64}\pi$
- `numZRanges_`: 64
- `numXRanges_`: 64
- `filterOutCenterR_`: 140.0
- `filterOutCenterZ_`: 550.0
- `hitsThresholds_`: 7 for every wedge
- `linesCrossingsThresholds_`: 3 for every wedge
- `LayerHitThreshold`: 3
- `PhiOMaxDivisionLevel`: 11
- `QOverPtMaxDivisionLevel`: 9
- 10 points per particle

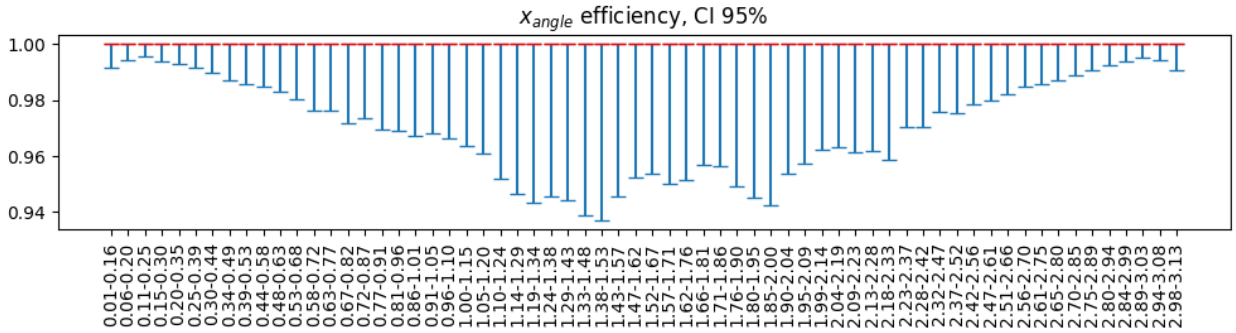


Figure 21: RPPI test: efficiency of finding solutions by x_{angle} range with 95% confidence intervals.

Figure 22 shows the efficiency for 64 equal-width z_{angle} bins. The test was run with the exact same settings as for the x_{angle} efficiency test. In this test, the solver also found all particles, demonstrating its ability to find helixes regardless of particle direction in the xy plane.

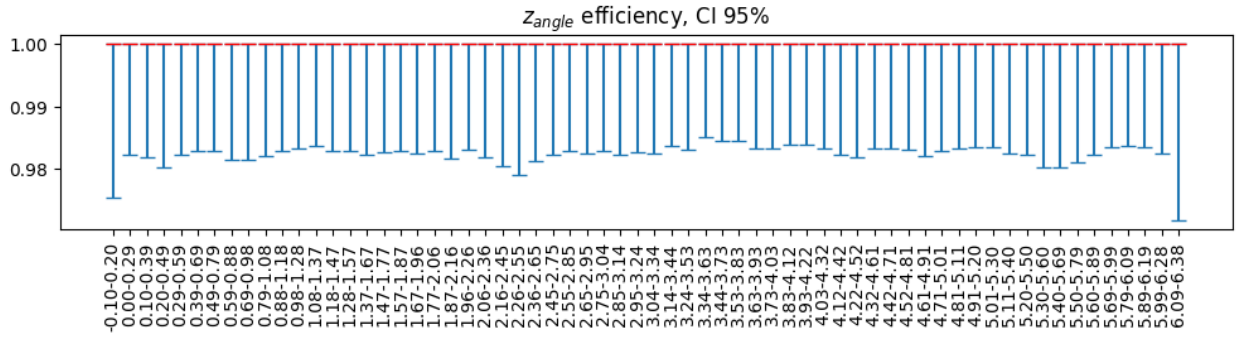


Figure 22: RPPI test: efficiency of finding solutions by z_{angle} range with 95% confidence intervals.

6.1.2 Efficiency by z-angle Margin

32 different z_{angle} margin values in the range $[0, \frac{31}{32} \cdot \frac{\pi}{16}]$ were used in separate RPPI tests. Figure 23 shows a plot of efficiency vs. z_{angle} margin. The tests were run with the exact same settings as for the x_{angle} and z_{angle} range efficiency tests except for the z_{angle} margin. As can be seen, efficiency reaches 1.0 for a z_{angle} margin of ≈ 0.1 rad, which corresponds to the $\frac{1}{32}\pi$ value used in the test. Other tests, some of them described below, indicated that optimal margins depend on the number of ranges used, and that efficiency improves with a decreasing number of ranges.

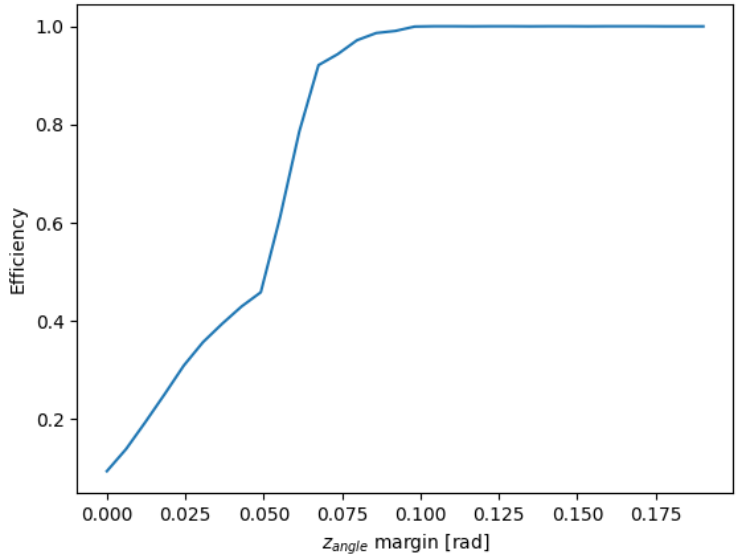


Figure 23: RPPI test: efficiency of finding solutions by z_{angle} margin.

6.1.3 Efficiency by x-angle Margin

Tests for x_{angle} margin were conducted in an analogous way with z_{angle} margin set to $\frac{1}{32}\pi$ and x_{angle} margin in the range $[0, \frac{31}{32} \cdot \frac{\pi}{64}]$. Figure 24 shows a plot of efficiency vs. x_{angle} margin. Efficiency reaches 1.0 for an x_{angle} margin of $\approx \frac{1}{128}\pi$.

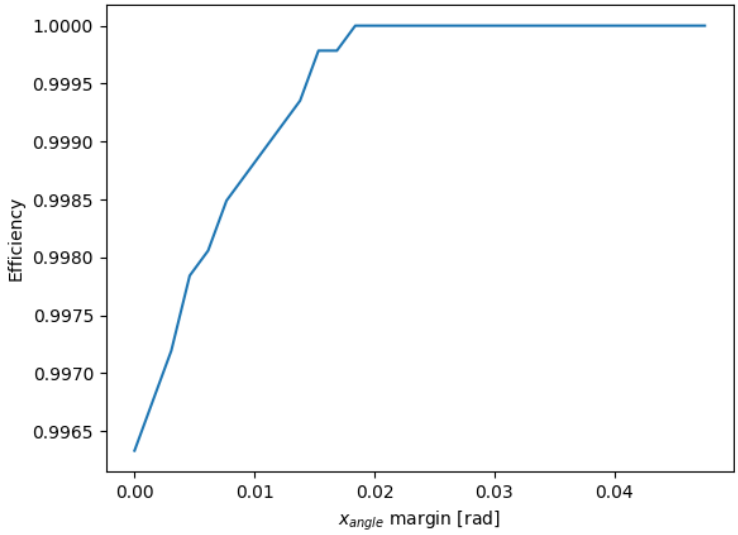


Figure 24: RPPI test: efficiency of finding solutions by x_{angle} margin.

6.1.4 Efficiency by Number of x-angle and z-angle Ranges

A number of tests were run with the same settings as for previous tests with the exception of x_{angle} and z_{angle} margins and range counts. The z_{angle} margin was set to $\frac{1}{32}\pi$ and the x_{angle} margin was set to $\frac{1}{128}\pi$. The x_{angle} and z_{angle} ranges were set in pairs so as to cover a grid of $[8, 128]$ with step 8 for both x_{angle} and z_{angle} ranges. Figure 25 shows the results of these tests as a heatmap. Increasing the number of z_{angle} ranges above 80 decreases efficiency noticeably, while increasing the number of x_{angle} ranges does not exhibit such an effect within the range of tested values.

The purpose of increasing range counts is to reduce the number of points processed per run of AHT and therefore decrease the workload. However, as the width of the region decreases, the ratio of points in the margins to the points in the main part of the region increases, which increases the number of points that need to be processed for more than one splitter region. Hence, increasing the number of divisions above certain values does not yield any benefit. The upper limit for the number of divisions is such that the entire non-margin part of a region n is covered by the margins of its neighbors $n-1$ and $n+1$. The optimal x_{angle} margin of $\frac{1}{32}\pi$ obtained in previous tests implies 64 as the sensible upper limit of regions in the xy plane. Similarly, the optimal z_{angle} margin of $\frac{1}{128}\pi$ implies 128 as the sensible upper limit of regions in the zy plane.

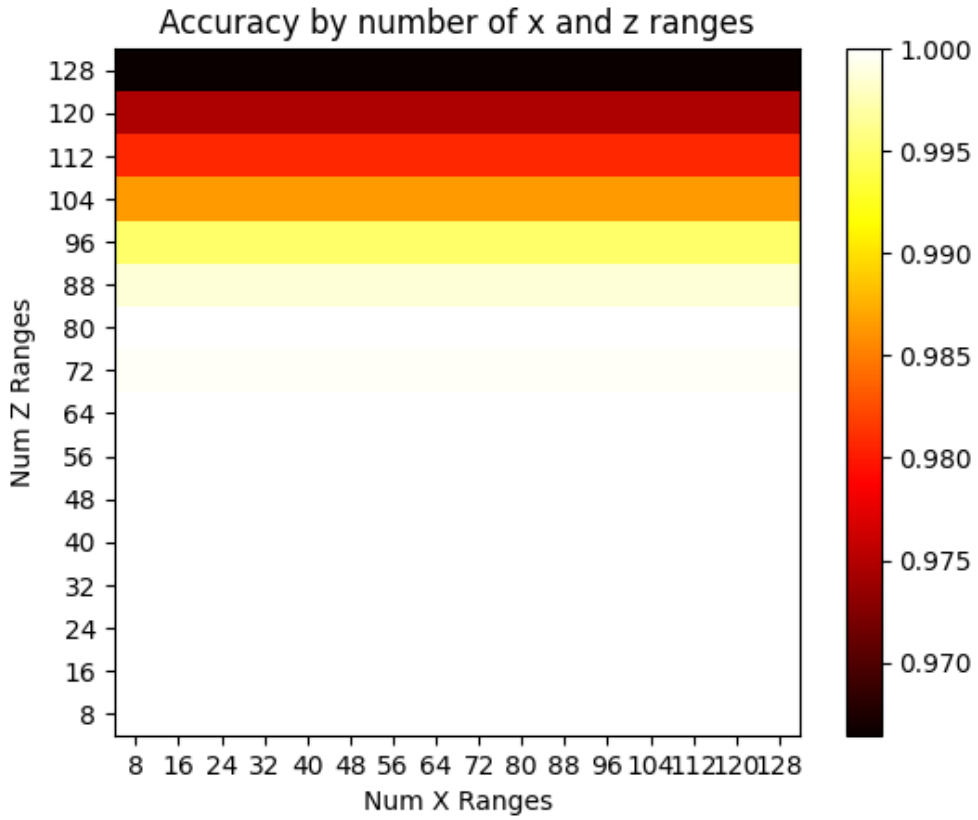


Figure 25: RPPI test: efficiency heatmap for x_{angle} and z_{angle} ranges.

6.2 Particles Initial

Spacepoints associated with all the particles participating in the tests are merged and processed in a single helix solver run. In contrast to RPPI, spacepoints of neighboring particles may interfere with each other's trajectory reconstruction. This test is helpful for narrowing down the range of suitable solver settings that allow for finding solutions. As for RPPI, this test used near-perfect helixes generated in the exact same way. Solution acceptance criteria are also the same as for RPPI tests.

Common settings for all PI tests are:

- `minZAngle_`: 0
- `maxZAngle_`: 2π
- `minXAngle_`: $\frac{5}{256}\pi$
- `maxXAngle_`: $\frac{251}{256}\pi$
- `poleRegionAngle_`: $\frac{5}{256}\pi$
- `zAngleMargin_`: $\frac{1}{32}\pi$
- `xAngleMargin_`: $\frac{1}{128}\pi$
- `filterOutCenterR_`: 140.0
- `filterOutCenterZ_`: 550.0
- `hitsThresholds_`: 7 for every wedge

- `linesCrossingsThresholds_`: 3 for every wedge
- `LayerHitThreshold`: 3
- `PhiOMaxDivisionLevel`: 11
- `QOverPtMaxDivisionLevel`: 9
- 10 points per particle

6.2.1 Efficiency by Number of x-angle and z-angle Ranges

The test setup for efficiency by x_{angle} and z_{angle} margins was the same as for the analogous RPPI test with the exception of range values. x_{angle} and z_{angle} ranges were set in pairs to cover a grid of $[8, 64]$ range with step 4 in both dimensions. Figure 26 shows a heatmap with efficiency values for x_{angle} and z_{angle} ranges.

For every range pair, efficiency is ≈ 1. Only for 48 x_{angle} ranges does efficiency differ, but the difference is caused by a single trajectory not found.

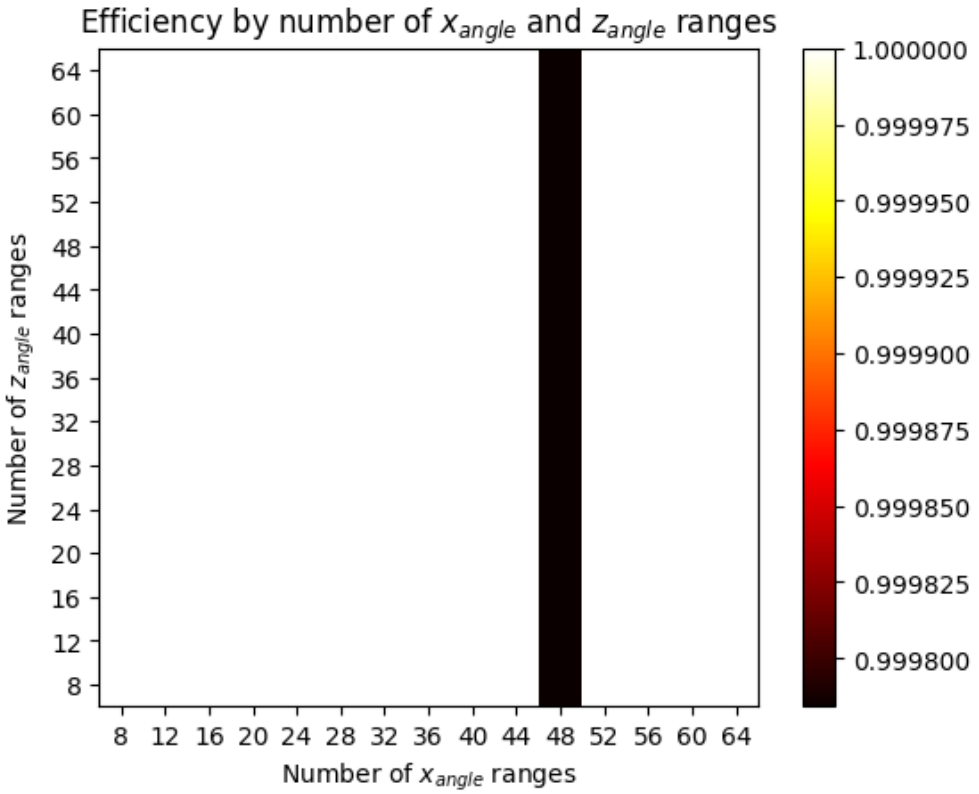


Figure 26: PI test: efficiency heatmap for x_{angle} and z_{angle} ranges.

6.2.2 Number of Solutions by Number of x-angle and z-angle Ranges

The test described above also measured the number of solutions for each range pair. Results are presented in Figure 27.

The number of solutions drops rapidly with the number of ranges in the x_{angle} dimension up to about 32 ranges. This can be explained by the fact that a narrower x_{angle} range results in a narrower x_{angle} range of particle trajectories in the same splitter region. As the respective spacepoints are projected onto the xy plane, trajectories of particles differing in x_{angle} are indistinguishable. They interfere with each other's solutions. Due to the nature of the Hough transform algorithm, the interference is always positive, thus creating false positives. A lower number of particles in the same AHT run results in less interference.

The number of solutions rises with the number of ranges in the z_{angle} dimension above 16 ranges due to less overlap between splitter regions. This suggests that the number of ranges should be $\geqslant 32$ in the x_{angle} dimension and $\leqslant 16$ in the z_{angle} dimension. However, the number of z_{angle} ranges impacts the final accumulator resolution. To counteract this effect, the division level in the ϕ_0 direction could be increased, which in turn would increase the number of solutions.

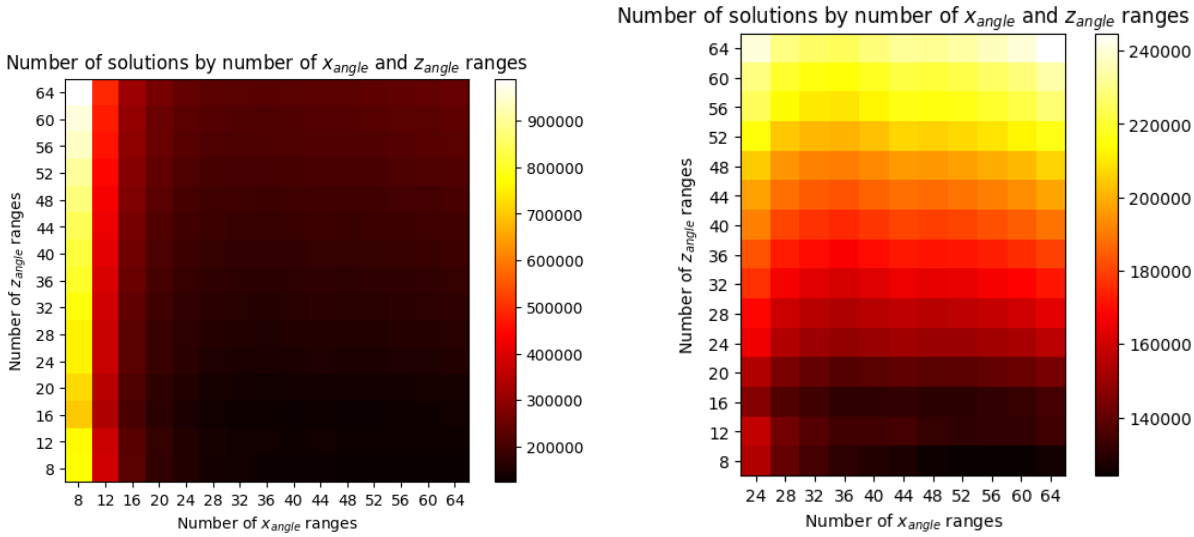


Figure 27: PI test: number of solutions heatmap for x_{angle} and z_{angle} ranges.

6.2.3 True Positive Rate by Number of x-angle and z-angle Ranges

To estimate the proportion of particles with successfully reconstructed trajectories, false positives, and redundant solutions, a true positive rate (TPR) metric was introduced. We define TPR as (35):

$$T = \frac{N_P E}{N_S} \tag{35}$$

where:

- T - TPR
- N_P - number of particles initial
- E - efficiency
- N_S - total number of solutions

TPR can be regarded as an estimate of the proportion of unique solutions truly reconstructing a trajectory. Additionally, $1/TPR$ may be regarded as an estimate of the combined duplication and false positives rate.

Since a particle trajectory is counted as reconstructed if there exists a matching solution, if two or more particles have very similar trajectories, it is possible that they share a solution. Hence, for efficiency ≈ 1 and little to no false positives and redundant solutions, TPR may be greater than 1, although this is a very rare case. In the presented results, TPR has been restricted to 1.

For PI and RPPI tests, spacepoints are generated using all the particles with $p_T \geqslant 1$ GeV/c, but for efficiency calculation, only those with $r < 0.1$ mm and $|z| < 200$ mm origin are used. This results in a solution search for 5040 particles with only 4633 of them participating in the efficiency calculation. As a consequence, TPR is slightly underestimated.

TPR was also calculated for each range pair in the test described above. Results are presented in Figure 28.

TPR rises with the number of ranges in the x_{angle} dimension with a peak between 48 and 60 ranges. TPR also drops with the number of ranges in the z_{angle} dimension with the same causes and implications as for the number of solutions. Having these results in mind, spacepoints full event tests described below were conducted with 48 ranges in the x_{angle} dimension and 32 ranges in the z_{angle} dimension.

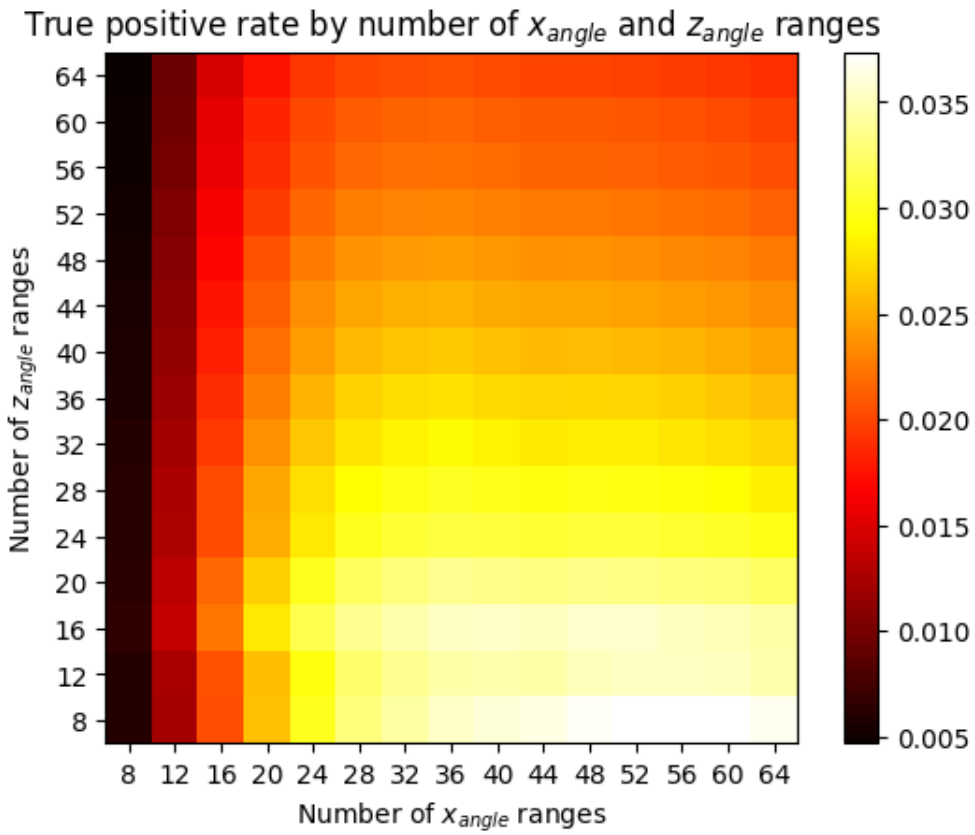


Figure 28: PI test: TPR heatmap for x_{angle} and z_{angle} ranges.

6.3 Spacepoints Full Event

Spacepoints full event tests use full sets of spacepoints from simulated events with pile-up $\mu = 200$. All spacepoints are processed together in a single helix solver run as they would be in a real use case scenario. This is also the closest possible test to a real application with the test data supplied for this project. Solution acceptance criteria are the same as for RPPI and PI tests.

Common settings for all Spacepoints tests are:

- `minZAngle_`: 0
- `maxZAngle_`: 2π
- `minXAngle_`: $\frac{5}{256}\pi$
- `maxXAngle_`: $\frac{251}{256}\pi$
- `poleRegionAngle_`: $\frac{5}{256}\pi$
- `zAngleMargin_`: $\frac{1}{32}\pi$
- `xAngleMargin_`: $\frac{1}{128}\pi$
- `numZRanges_`: 32
- `numXRanges_`: 48
- `filterOutCenterR_`: 140.0

- `filterOutCenterZ_`: 550.0
- `LayerHitThreshold`: 3
- `Phi0MaxDivisionLevel`: 11
- `QOverPtMaxDivisionLevel`: 9

6.3.1 Efficiency, Number of Solutions, and TPR Sensitivity to Hits Threshold and Lines Crossing Threshold by x-angle Range

A test was conducted to explore the impact of hits threshold (`hitsThresholds_`) and lines crossing threshold (`linesCrossingsThresholds_`) on efficiency, number of solutions, and TPR. For hits threshold values in the $[5, 10]$ range and lines crossing threshold values in the $[3, 5]$ range, the helix solver was run with 20 distinct events and results were averaged. Figure 29, Figure 30, and Figure 31 show efficiency results broken down by x_{angle} range in the form of heatmaps. Note that for efficiency, the color scale was adjusted for better readability. Values below 0.9, 0.8, and 0.5 are clamped to 0.9, 0.8, and 0.5 for consecutive heatmaps as a side effect.

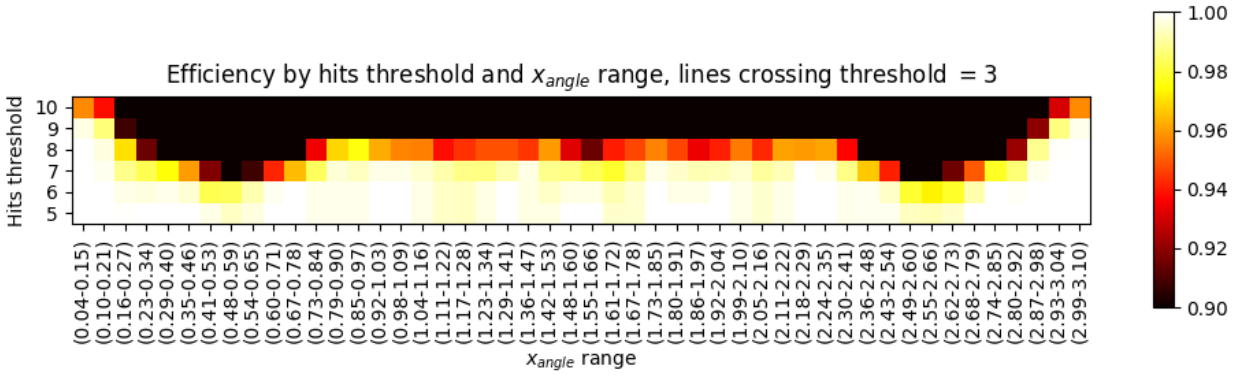


Figure 29: S test: efficiency heatmaps for hits threshold and x_{angle} ranges with lines crossing threshold $= 3$.

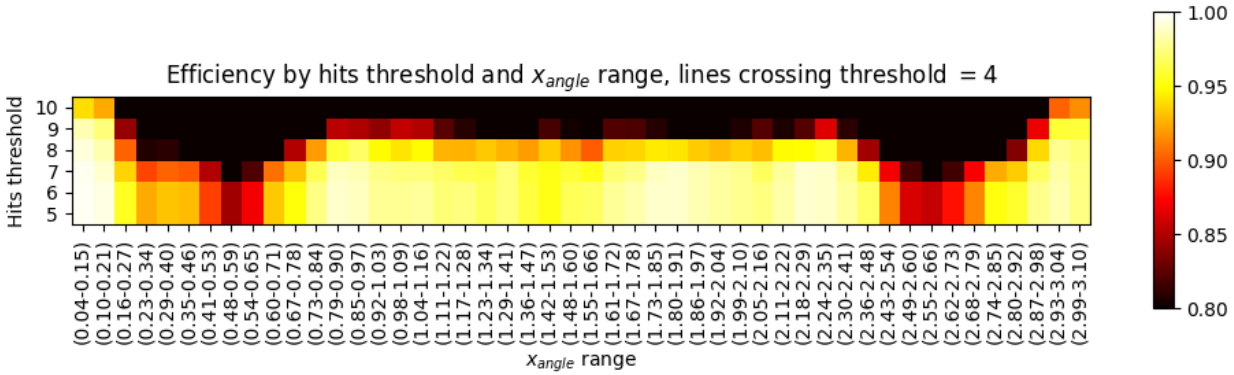


Figure 30: S test: efficiency heatmaps for hits threshold and x_{angle} ranges with lines crossing threshold $= 4$.

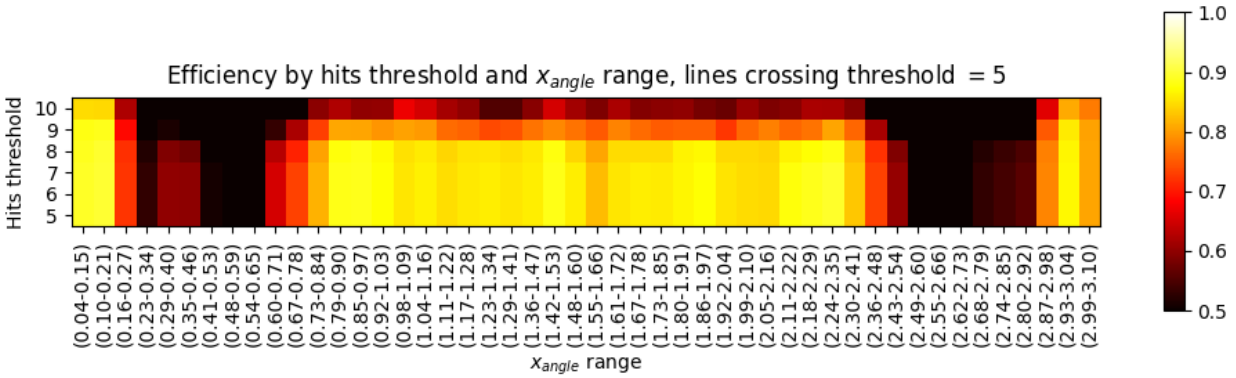


Figure 31: S test: efficiency heatmaps for hits threshold and x_{angle} ranges with lines crossing threshold = 5.

Figure 32, Figure 33, and Figure 34 show numbers of solutions as heatmaps. Note that scale is logarithmic.

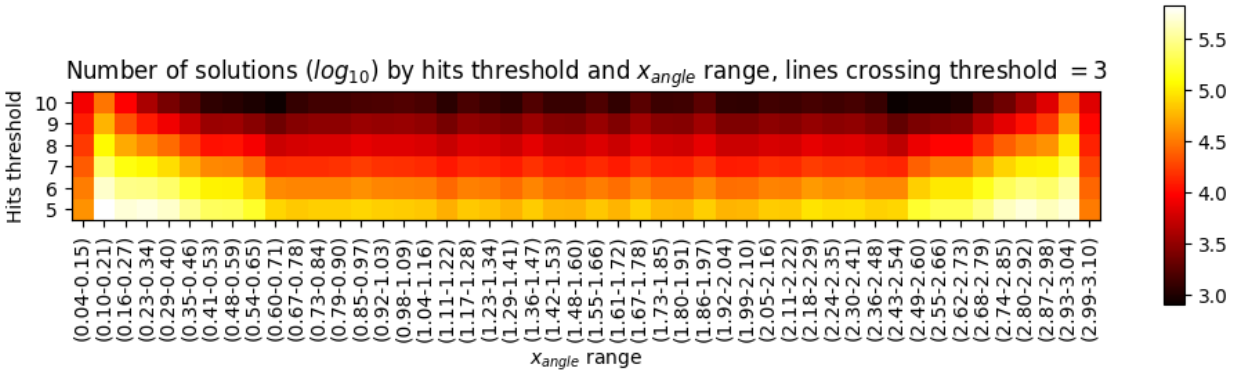


Figure 32: S test: number of solutions heatmaps for hits threshold and x_{angle} ranges with lines crossing threshold = 3.

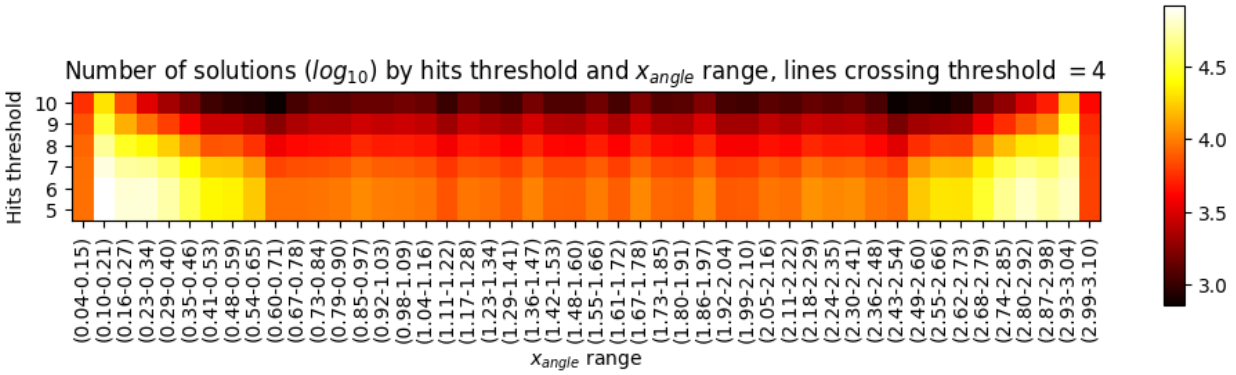


Figure 33: S test: number of solutions heatmaps for hits threshold and x_{angle} ranges with lines crossing threshold = 4.

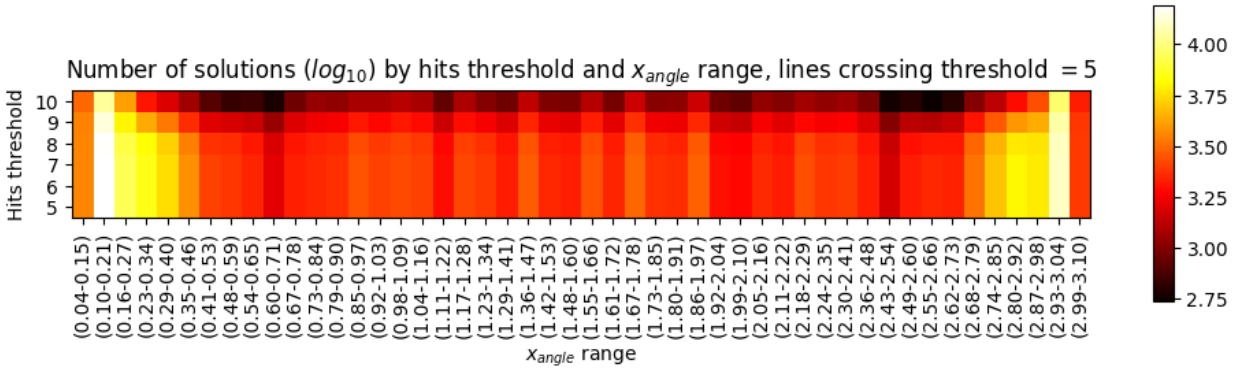


Figure 34: S test: number of solutions heatmaps for hits threshold and x_{angle} ranges with lines crossing threshold = 5.

Figure 35, Figure 36, and Figure 37 show true positive reate heatmaps.

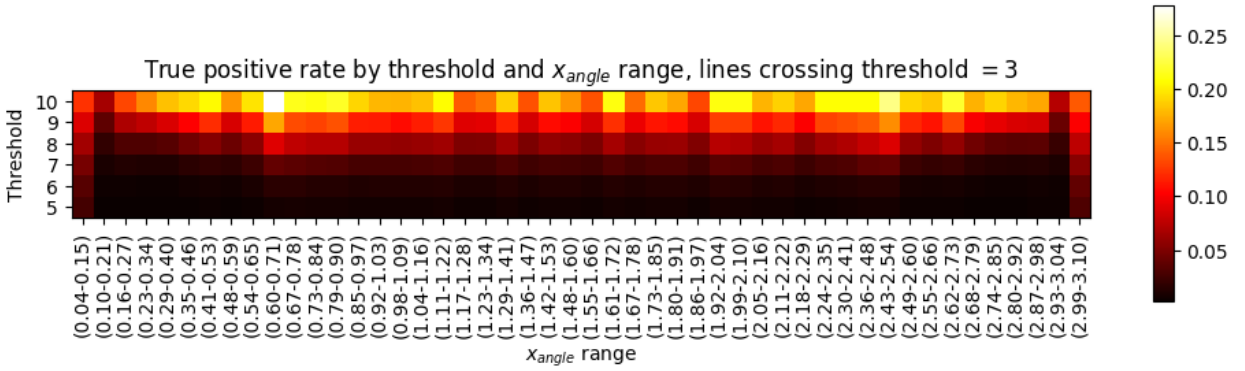


Figure 35: S test: TPR heatmaps for hits threshold and x_{angle} ranges with lines crossing threshold = 3.

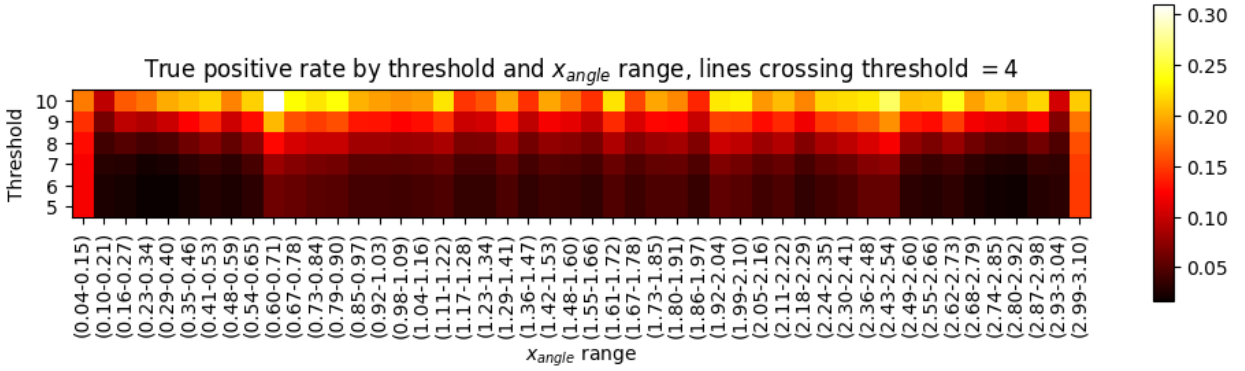


Figure 36: S test: TPR heatmaps for hits threshold and x_{angle} ranges with lines crossing threshold = 4.

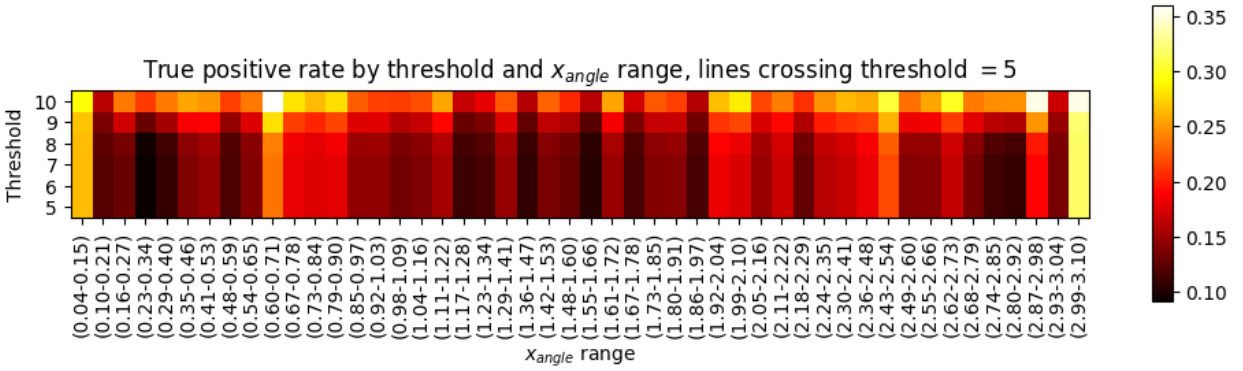


Figure 37: S test: TPR heatmaps for hits threshold and x_{angle} ranges with lines crossing threshold $= 5$.

As the heatmaps show, all the measured quantities exhibit a strong response to hits thresholds and lines crossing thresholds. Results vary significantly depending on the x_{angle} range. Such behavior allows for a trade-off between efficiency, TPR, and inherently execution time. The efficiency heatmaps suggest thresholds no lower than 7 for the central x_{angle} region, $\geqslant 7$ for extremes, and 6 for regions in between.

6.3.2 Full Event Test for Selected Per-Region Thresholds

This test was created for demonstrative purposes. It shows the solver's characteristics for selected per-region thresholds. The lines crossing threshold was set to 4 for all regions. The hits threshold was set according to the Table 3. The test was run with 20 distinct events and results were averaged.

Region index	0	1	2	3	4	5 - 10	11 - 20	21 - 26	27	28	29	30	31
Hits threshold	9	9	8	8	7	6	7	6	7	8	8	9	9

Table 3: Hits thresholds for each region.

Figure 38 shows the number of particles initial for each x_{angle} range. Figure 39 shows the efficiency for each x_{angle} range. Figure 40 shows the number of solutions for each x_{angle} range.

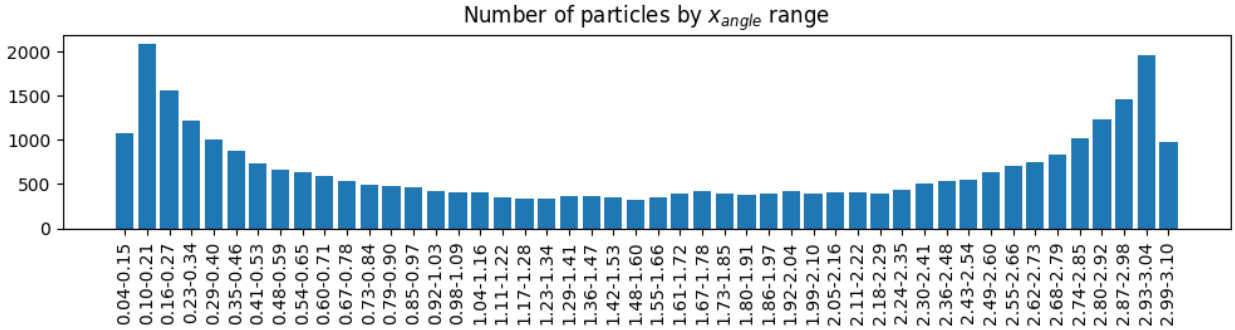


Figure 38: S test: Number of particles initial for each x_{angle} range.

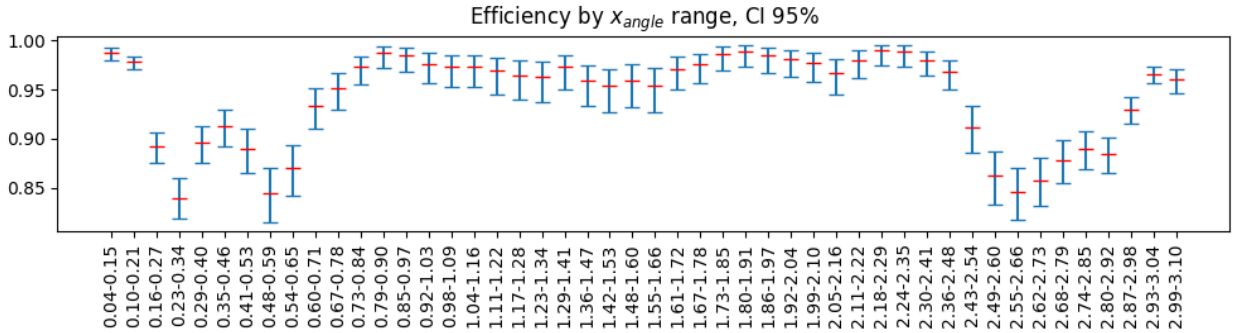


Figure 39: S test: efficiency for custom hits thresholds and lines crossing threshold 4.

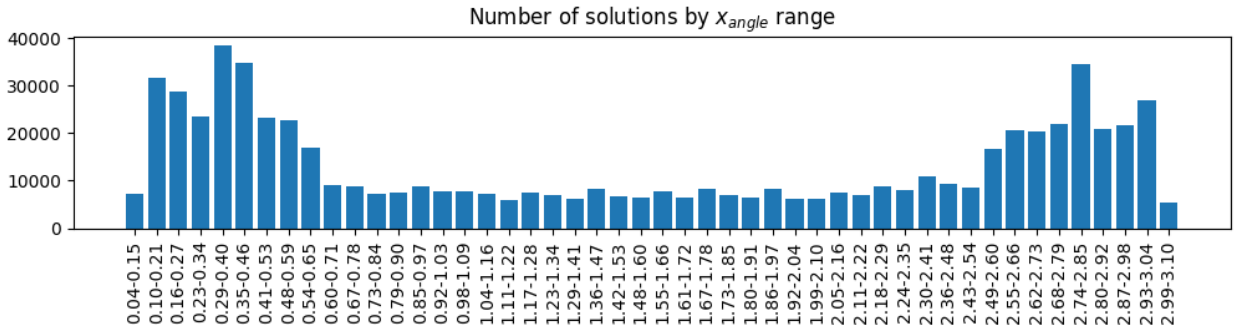


Figure 40: S test: number of solutions for custom hits thresholds and lines crossing threshold 4.

Figure 41 shows the number of particles initial for each p_T range. Columns colored in red mark no particles initial in the respective p_T range. Figure 42 shows the efficiency for each p_T range. Confidence intervals in red mark no particles initial in the respective p_T range.

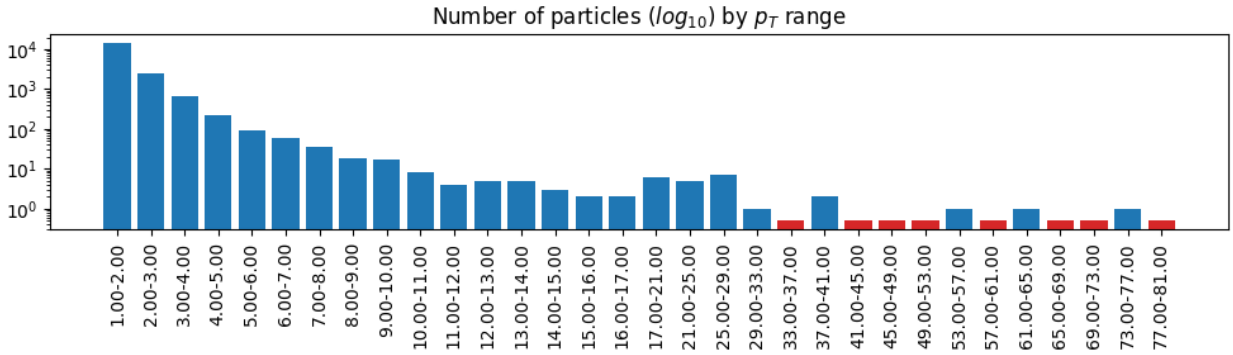


Figure 41: Number of particles initial for each p_T range.

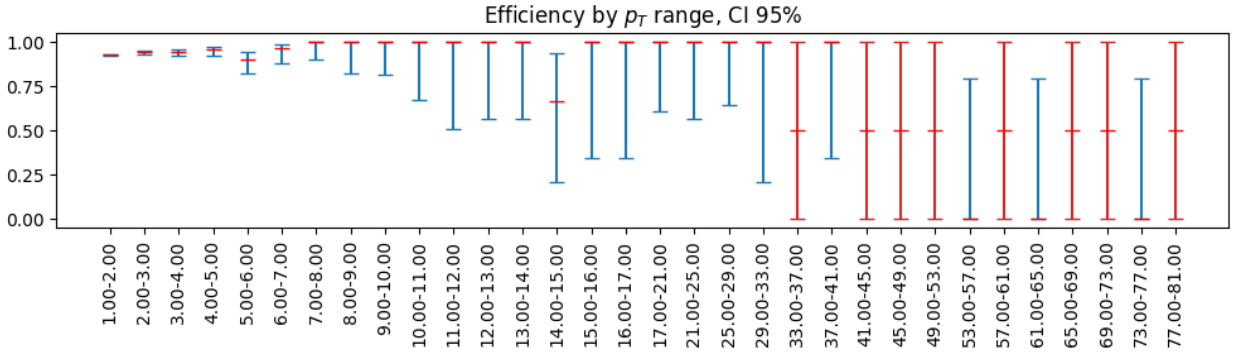


Figure 42: S test: efficiency for custom hits thresholds and lines crossing threshold 4 by p_T range.

6.3.3 Accidentally Correct Solutions Rate

Due to the high number of solutions, many of them match with actual trajectories by chance. To estimate the rate of such cases, a custom test was designed. The charge of every solution was negated, rendering them unable to match with their respective particles. On the other hand, they became able to match with particles of the opposite charge, which is only possible if other parameters considered for matching align by chance. Figure 43 shows the results of the mentioned test broken down by x_{angle} range. Test settings were the same as for the subsubsection 6.3.2 with selected per-region thresholds.

This test, besides providing insight into the accidentally correct solutions rate, also proves to some extent that the achieved efficiency rates are not merely a result of a high number of solutions or faulty implementation artifacts.

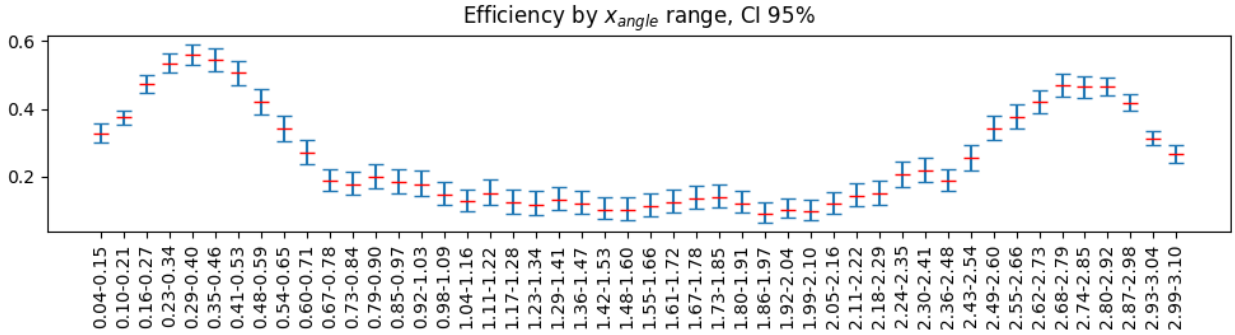


Figure 43: S test: accidental efficiency for custom hits thresholds and lines crossing threshold 4.

6.3.4 Characteristics of Non-Reconstructed Trajectories

The characteristics of non-reconstructed trajectories were investigated to find potential correlations with particle parameters. For this purpose, four plots were created:

- v_x vs. v_y or particle origin position in the xy plane
- v_z vs. v_r or particle origin position in the zr plane projection
- $\log_{10}(p)$ vs. $\log_{10}(p_T)$
- η vs. $\log_{10}(p_T)$

Test settings were the same as for the subsubsection 6.3.2 with selected per-region thresholds.

Results are shown in Figure 44. As can be seen, there is no visible correlation between particle origin parameters v_x, v_y, v_z, v_r and the probability of trajectory reconstruction for the particle cluster created within the interaction region as described in the subsection 3.3 regarding particle and trajectory characteristics. There is, however, a significant drop in the probability of trajectory reconstruction for particles created outside of the interaction region with origin v_r above some threshold (not shown in the plots), which is likely caused by violating assumptions for the interaction point made in the subsubsection 4.1.3 explaining the derivation of Hough transform equations used in the AHT. Plots for p vs. p_T and η vs. p_T show a clear correlation between η (or x_{angle}) and the probability of trajectory reconstruction with two lows positioned symmetrically from 0.

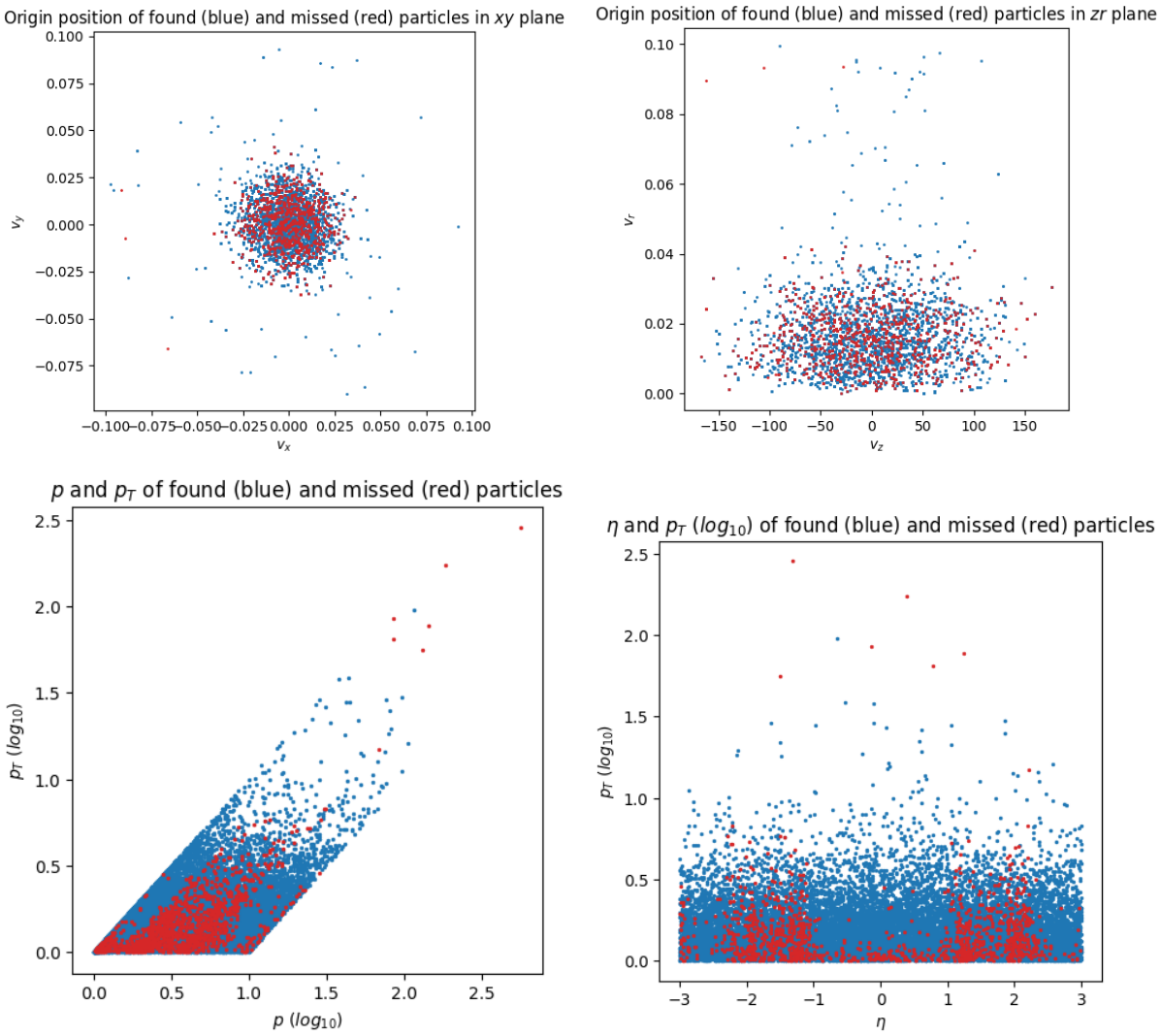


Figure 44: S test: particle characteristics and trajectory reconstruction correlations.

7 CPU Performance Tests

Performance tests were conducted to evaluate the execution times of implemented solution for various settings. Unfortunately, due to limited resources and implementation constraints imposed by OneAPI usage in the project, results are biased by multiple factors. As a consequence, results should not be used for fine-grained comparisons and analyses. However, they can be used to get a general idea of the solver's performance and responsiveness to settings. The list of factors that may have an impact on results and general test environment characteristics is as follows:

- Tests were run on an 8th-generation Intel Xeon E-2176M CPU, a laptop processor with 6 physical cores, 12 threads, $64kB$ of L1 cache, $256kB$ of L2 cache, $12MB$ of L3 cache, and a $2.7GHz$ base frequency.
- Every event was processed in a separate thread. Threads were running in parallel in groups of 10.
- The executable was compiled with the clang/LLVM 19.0.0 compiler.
- The executable was compiled with, among many others, the following flags: `-O3 -pthread -fsycl -fsycl-targets=nvptx64-nvidia-cuda`.
- Tests were running in a Docker container in interactive mode with the following flags: `(docker run -gpus all -rm -it -security-opt seccomp=unconfined helix-solver-docke`
- Hardware was overheating, resulting in CPU throttling, which biased results for tests not at the beginning of the test queue. To mitigate this effect for heatmap tests, the computer was preheated for at least 15 minutes by running dummy tests before the actual ones.
- Division levels are the same for every test, which biases results as the final resolution decreases with a decreasing number of splits in the z_{angle} direction.

Common settings for all performance tests are:

- `minZAngle_`: 0
- `maxZAngle_`: 2π
- `minXAngle_`: $\frac{5}{256}\pi$
- `maxXAngle_`: $\frac{251}{256}\pi$
- `poleRegionAngle_`: $\frac{5}{256}\pi$
- `zAngleMargin_`: $\frac{1}{32}\pi$
- `xAngleMargin_`: $\frac{1}{128}\pi$

- `filterOutCenterR_`: 140.0
- `filterOutCenterZ_`: 550.0
- `linesCrossingsThresholds_`: 4 for every wedge
- `LayerHitThreshold`: 3
- `Phi0MaxDivisionLevel`: 11
- `QOverPtMaxDivisionLevel`: 9

Execution times do not include time spent on data loading and memory allocation, as memory can be allocated in advance and reused for subsequent helix solver runs.

7.1 Execution Times by Number of x-angle and z-angle Ranges

To evaluate the impact of the number of x_{angle} and z_{angle} ranges on execution time, a test was conducted with the same 20 events for every configuration and results were averaged. Figure 45 shows execution times in three heatmaps presenting total execution time, splitter execution time, and solver execution time.

Tests were run with `hitsThresholds_` $= 7$ for every wedge.

As can be seen, execution time for the splitter rises with the number of splits in both x_{angle} and z_{angle} directions. The current splitter implementation has similar Θ and O complexity, which is as follows (36):

$$O(NZX) \tag{36}$$

where:

- N - number of spacepoints
- Z - number of z_{angle} ranges
- X - number of x_{angle} ranges

However, due to the shape of wedges, the splitter may be optimized to have $O(N)$ complexity. This was not attempted, as the splitter run usually accounts for less than 10% of the total execution time. Also, splitter optimization is a very low priority for this work.

Execution times for the solver are not as conclusive as for the splitter due to the reasons listed at the beginning of this chapter. Due to the changing actual resolution of AHT, we cannot draw conclusions except that the optimal number of splits from an execution time point of view in the z_{angle} direction is not above 8. For the x_{angle} direction, the optimal number of ranges seems to be around 20. However, this comes at the cost of a higher number of false positives. Values below 20 are not optimal from both performance and results quality points of view.

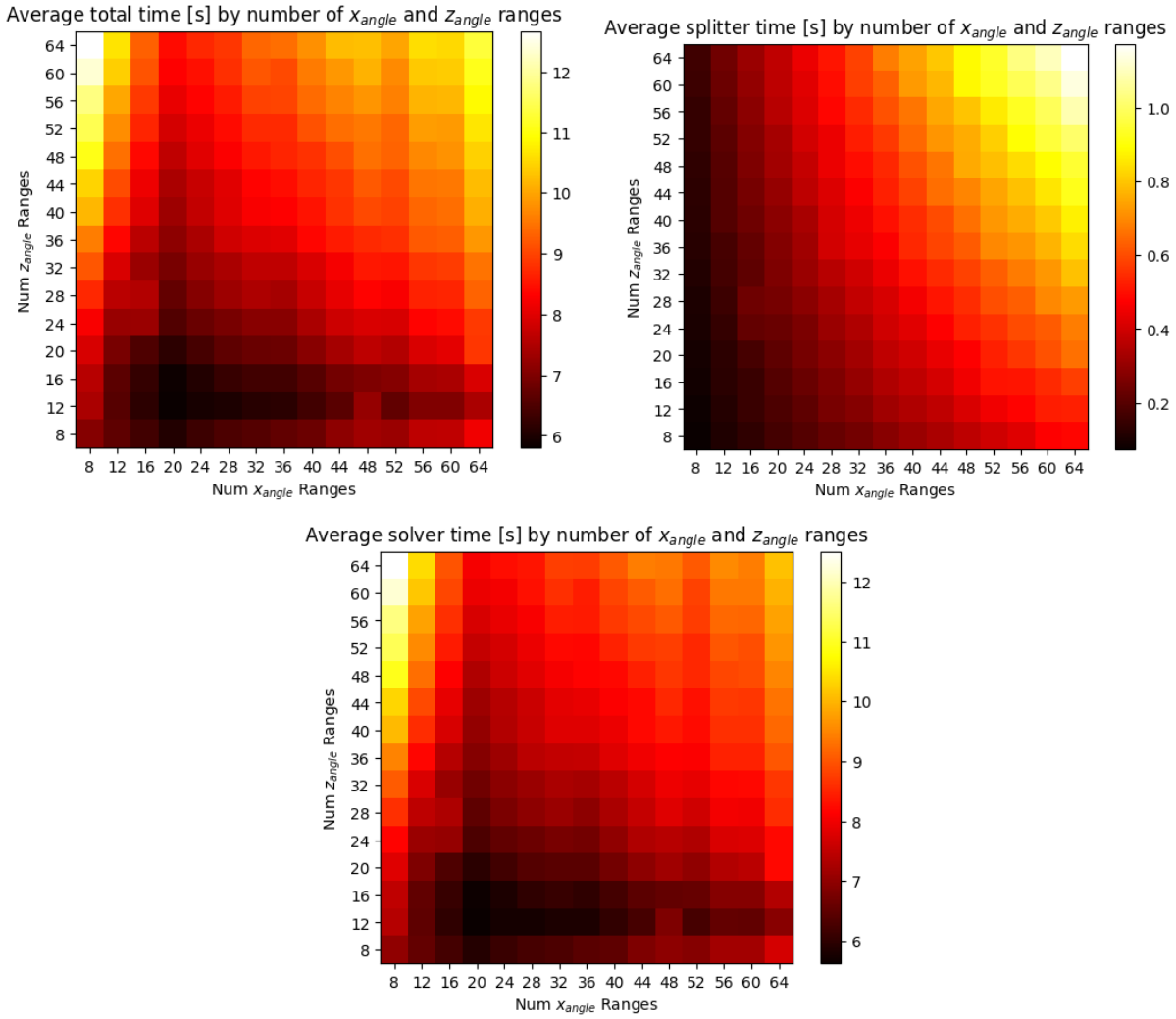


Figure 45: S test: execution times by number of x_{angle} and z_{angle} ranges.

7.2 Execution Times for Custom Per-Region Hits Thresholds

Two tests were run to check execution times for the solver with settings as used in the subsubsection 6.3.2. One test was run on pre-heated hardware and the other after it cooled down. Both tests were run with 20 distinct events and repeated 5 times. Averaged results are presented in Table 4.

Pre-heated / cooled-down	Total execution time	Splitter execution time	Solver execution time
Pre-heated	8.55 s	0.59 s	7.97 s
Cooled-down	8.24 s	0.57 s	7.67 s

Table 4: Execution times for custom per-region hits thresholds on pre-heated and cool hardware.

8 GPU Implementation

8.1 Technical Challenges Due to OneAPI/SYCL and CUDA

One of the goals of the project was to create a GPU implementation that would make full use of resources and hardware capabilities, and to base this implementation on OneAPI/DPC++ (SYCL) technology. The use of these technologies was associated with many difficulties, particularly those resulting from the need to reconcile OneAPI/DPC++ with CUDA, which determined the possibility of offloading to Nvidia GPUs. The following list highlights some of these problems:

- Intel OneAPI does not currently have native support for Nvidia GPUs. Creating an implementation capable of offloading to Nvidia GPUs requires the use of CUDA. This combination is problematic because it requires adapting the implementation to the restrictions imposed by OneAPI/DPC++ and CUDA simultaneously.

- At the time of creating the implementation for GPUs, there was no official support for OneAPI based on CUDA from Intel. The only solution suggested by Intel was to use a plugin provided by Intel subsidiary - Codeplay.

- Using the combination of OneAPI, CUDA, and the Codeplay plugin raised multiple issues related to configuration and environment setup, especially in the face of changing software versions and their varying compatibility. To solve these problems, a Docker image was created containing a configured environment with frozen software versions. All work was carried out in a container running in interactive mode.

- The combination of these technologies resulted in severe limitations on the availability of information about runtime errors and often led to errors being covered up, causing debugging efforts to be misdirected.

- Executing code on a GPU significantly hinders, and often completely prevents, the use of debugging tools.

- The SYCL standard explicitly prevents memory allocation on the heap inside kernel code (parts of the program offloaded to the accelerator) [Rov+20]. This forces allocation of all the used memory on the stack or pre-allocation before kernel execution. CUDA, on the other hand, limits the amount of thread memory allocated on the stack to $512kB$ [25]. This is definitely not enough for our needs. Therefore, memory for all data processed by the solver and for the results must be allocated in advance by instructions executed on the host side and transferred as pointers to the kernel before its execution.

- AHT is inherently recursive. Recursion in code offloaded to the accelerator is explicitly prohibited by the SYCL standard [Rov+20]. Therefore, an iterative approach is required.

- Due to the inability to use recursion and dynamic memory allocation, it is impossible to use STL containers and a significant part of the standard library.

8.2 Performance Issues

The GPU implementation is burdened with problems related to performance and efficient use of hardware resources. Since test data and results must be transferred between the host and the device (accelerator), it is crucial to plan transfers well so that they do not become a bottleneck. However, a much bigger problem for performance is frequent execution path branching and its consequences, as described in the section 4.4 on other AHT characteristics.

8.3 GPU Implementation Abandonment

The GPU implementation has been brought to a state where initial performance and quality tests can be conducted. Numerous issues related to debugging, very slow progress, and the difficulty of making changes significantly slowed down the development and testing of the solver's characteristics. In addition, the execution time on the GPU was usually around $200s$ despite parallel processing of all splitter regions, while processing an event on a single CPU thread took around $5s$ at the time. Due to the above factors and the fact that the algorithm is deterministic and the results are the same for CPU and GPU, further work on the development of the GPU implementation was abandoned.

8.4 GPU Implementation Design

Due to the fact that the GPU implementation was an important part of the project and accounted for an even greater share of the workload, its design is concisely described in this chapter despite the abandonment of further work on its development.

8.4.1 Data Organizing Objects

The processed data and results have been organized into two classes for easy and efficient processing and transfer: `EventUsm` and `ResultUsm`. In order to optimize transfers between the host and the device, memory management for this data was performed manually. These classes have their counterparts for the implementation on the CPU.

`EventUsm` is a class representing a single event. An object of this class stores raw data for all event measurements, their quantity, and ID. Objects contain pointers to data on both the host and device sides.

`ResultUsm` is a class representing the results for a single event. An object of this class stores the results for all splitter regions, their quantity, and ID. Objects contain pointers to data on both the host and device sides.

The `EventUsm` and `ResultUsm` classes have common methods responsible for allocation, deallocation, and data transfers between the host and the device in a way that minimizes the volume of transferred data.

8.4.2 Splitter and Solver

The splitter was implemented in a way that allows it to be used in both CPU and GPU solutions. However, it is used differently, and sharing applies only to some functions.

The helix solver was initially identical for both implementations, except for the way the allocated memory was organized. After abandoning work on the GPU implementation, the structure of the solvers diverged, but the general design is still similar.

8.4.3 KernelMemory

`KernelMemory` is the base class for classes that hold data allocated on the device. It is mainly used to enforce uniform implementation and helps to avoid overlooking key memory management elements by using the template method design pattern.

8.4.4 TaskUsm

`TaskUsm` stores information about the current processing status and assigned resources. The class has methods that perform successive stages of processing in dedicated threads, along with partial validation of calls. `TaskUsm` also uses the template method design pattern with the assumption that derived classes will only need to override two methods responsible for handling received objects wrapping the memory allocated on the device and calling appropriate kernels in the correct order.

8.4.5 QueueUsm

`QueueUsm` is a wrapper for the `sycl::queue` object and aggregates resources associated with this object. This class is used to facilitate resource organization, management, and control of their use.

8.4.6 WorkerUsm

`WorkerUsm` operates on a single `QueueUsm` object, assigns its resources to `TaskUsm` objects, whose execution it manages by calling appropriate methods and responding to changes in the state of these objects. `WorkerUsm` also verifies the validity of `TaskUsm` objects at various stages of their processing.

8.4.7 IWorkerControllerUsm

`IWorkerController` is a single-method interface devised as a callback for `WorkerUsm` to notify about the completion of `TaskUsm`s.

8.4.8 HelixSolverTask

`HelixSolverTask` is a subclass of `TaskUsm` and, as intended, handles assigned resources and organizes the order of calls to the three kernels responsible for processing an event. These are, in order:

- `SingleRegionKernel`: responsible for processing a single splitter region. It contains both the splitter and solver. Processing takes place in parallel for all regions.
- `SumRegionNumSolutionsKernel`: a very simple kernel responsible for processing information regarding the number of solutions from individual splitter regions in order to enable parallel invocation of `ConcentrateSolutionsKernel`.
- `ConcentrateSolutionsKernel`: rewrites solutions from individual regions to a single, continuous memory area in order to minimize the volume of data transfer to the host. Processing takes place in parallel for all regions.

8.4.9 Single Allocation Policy and Resource Borrowing

Due to the fact that memory must be allocated on the device before kernel execution, memory management was optimized for execution time by allocating memory for all regions at the initialization of the program, before processing any events. This way, no memory allocation occurs during event processing. To achieve this, all memory resources are "borrowed" by tasks just before processing and returned immediately after. Memory is cleared before reuse to the lowest extent sufficient to avoid state leaks.

9 Final Considerations for Future Work

9.1 Multidimensional Accumulator

The accumulator used in this project is two-dimensional, which, as mentioned, limits the range of parameters of the particles found. If a three-dimensional accumulator (v_x, v_y, v_r) were used, it would also be possible to find particles created outside the interaction region.

The splitter was introduced for two purposes. The first was to divide the accumulator area according to the angle x_{angle}, so that fewer points are projected onto the xy plane in individual AHT runs, which significantly reduces the number of false and duplicate solutions. The second purpose was to reduce computational complexity. The assignment of points to individual intervals of the z_{angle} angle with margins is essentially identical to running AHT for the first division levels. Using a splitter for this purpose is optimal because it can be implemented with complexity $O(N)$, where N is the number of points. Furthermore, the processing of individual points by the splitter is independent, thus enabling parallelization and offering great potential for low-level optimization. The use of a 4-dimensional or 5-dimensional accumulator allowing for full parameterization of the helix (v_x, v_y, v_r, $pitch$, $phase$) would eliminate the need to use a splitter for the first of the above-mentioned purposes (although it would still benefit performance). In addition, the number of false and duplicate solutions would be significantly reduced.

The impact of more dimensions on performance is difficult to determine. On one hand, the number of cells to be processed after each division would double with every additional dimension. On the other hand, processing would converge much faster due to the earlier rejection of points that do not fit the solution.

9.2 Further Optimizations

The algorithm allows for the easy introduction of additional filters at the cell evaluation stage. The two filters used in the project definitely do not exhaust the possibilities.

Since in the final stages of division the number of points intersecting an accumulator cell is very small, it might be optimal to divide the cell at a certain division level into a number of parts corresponding to the final resolution and perform an analysis similar to the classical Hough transform in order to save time on making divisions and creating lists of points.

Due to the processing of cells in a manner similar to depth-first search, solutions from neighboring cells, and thus with similar parameters, are usually stored close to each other in memory. Therefore, it is possible to introduce a fast mechanism for duplicate reduction.

In the chapter 5.2.5 describing the algorithm, the rejection of points from the innermost

layers of the detector was described. Despite the high density of points, which makes their use difficult, these layers account for a significant part of the data and are a source of information that potentially can be used in some way to improve the quality of the results.

The implementation in general leaves a lot of space for low-level optimization.

References

[22] "ATLAS moves into top gear for Run 3". In: (2022). https://home.cern/news/news/physics/atlas-moves-top-gear-run-3.

[23] *ATLAS Event Display: HL-LHC simulated top-antitop quark pair production*. https://cds.cern.ch/images/ATLAS-PHOTO-2023-003-2. 2023.

[25] *CUDA C Programming Guide*. https://docs.nvidia.com/cuda/cuda-c-programming-guide/. 2025.

[Ai+22] Xiaocong Ai et al. "A Common Tracking Software Project". In: *Computing and Software for Big Science* 6.1 (2022). https://link.springer.com/article/10.1007/s41781-021-00078-8, p. 8. DOI: 10.1007/s41781-021-00078-8.

[Ale+08] M. Aleksa et al. "Measurement of the ATLAS solenoid magnetic field". In: *JINST* 3 (2008). https://cds.cern.ch/record/1073171, P04003. DOI: 10.1088/1748-0221/3/04/P04003.

[Apo+15] G. Apollinari et al. *High Luminosity Large Hadron Collider HL-LHC*. Tech. rep. CERN-2015-005. https://lss.fnal.gov/archive/design/fermilab-design-2015-02.pdf. CERN, 2015. DOI: 10.5170/CERN-2015-005.1.

[ATL23] Sebastien Roy-Garand on behalf of the ATLAS ITk collaboration. *ATLAS Inner Tracker Upgrade*. Tech. rep. ATL-ITK-PROC-2023-011. https://cds.cern.ch/record/2869708/files/ATL-ITK-PROC-2023-011.pdf. CERN, 2023.

[DH72] Richard O. Duda and Peter E. Hart. "Use of the Hough transformation to detect lines and curves in pictures". In: *Communications of the ACM* 15.1 (1972). https://dl.acm.org/doi/10.1145/361237.361242, pp. 11–15. DOI: 10.1145/361237.361242.

[Gaz24] Evangelos N. Gazis. "ATLAS Muon Spectrometer Upgrade for the HL-LHC Era's Challenges". In: *Symmetry* 16.8 (2024). https://www.mdpi.com/2073-8994/16/8/1035, p. 1035. DOI: 10.3390/sym16081035.

[Hou62] Paul V. C. Hough. "Method and means for recognizing complex patterns". US3069654A. https://patents.google.com/patent/US3069654. 1962.

[IK87] J. Illingworth and J. Kittler. "The Adaptive Hough Transform". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 9.5 (1987). https://ieeexplore.ieee.org/document/4767964, pp. 690–698. DOI: 10.1109/TPAMI.1987.4767964.

[Luc23] Claudio Luci. *The ATLAS experiment Phase-I upgrades for the LHC Run-3*. Tech. rep. ATL-UPGRADE-SLIDE-2023-510. https://cds.cern.ch/record/2871857. CERN, 2023.

[Rov+20] Maria Rovatsou et al. *SYCL 2020 Specification*. Tech. rep. Revision 10, https://registry.khronos.org/SYCL/specs/sycl-2020/html/sycl-2020.html. Khronos Group, 2020.

[VM15] Xabier Cid Vidal and Ramon Cid Manzano. *Taking a closer look at LHC: Parameters*. https://www.lhc-closer.es/taking_a_closer_look_at_lhc/1.lhc_parameters. 2015.

[Wil27] Edwin B. Wilson. "Probable Inference, the Law of Succession, and Statistical Inference". In: *Journal of the American Statistical Association* 22.158 (1927). https://www.tandfonline.com/doi/abs/10.1080/01621459.1927.10502953, pp. 209–212. DOI: 10.1080/01621459.1927.10502953.